QUT Centre for Robotics



ENN583 – Foundations of Robotic Vision

Sem 2, 2026

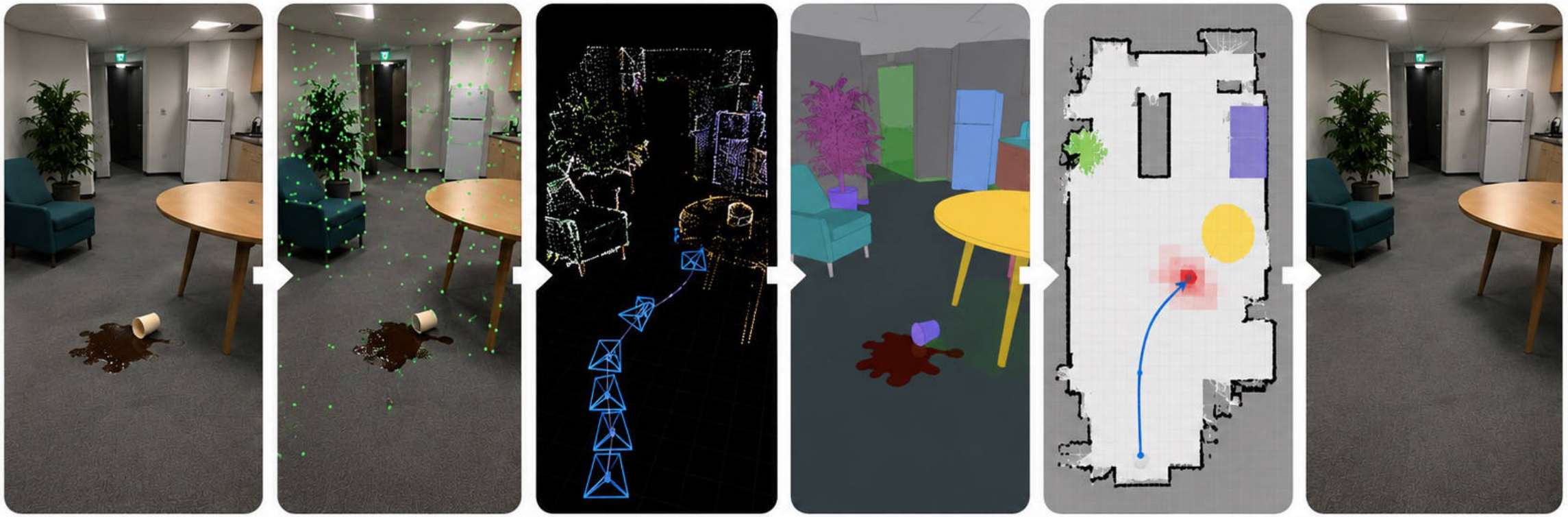
Prof Niko Suenderhauf

ACKNOWLEDGEMENT OF TRADITIONAL OWNERS

QUT acknowledges the Turrbal and Yugara, as the First Nations owners of the lands where QUT now stands. We pay respect to their Elders, lores, customs and creation spirits. We recognise that these lands have always been places of teaching, research and learning.

QUT acknowledges the important role Aboriginal and Torres Strait Islander people play within the QUT community.

What is Robotic Vision?



From pixels to information to actions.

CRICOS No.00213J

Let’s start with an example!

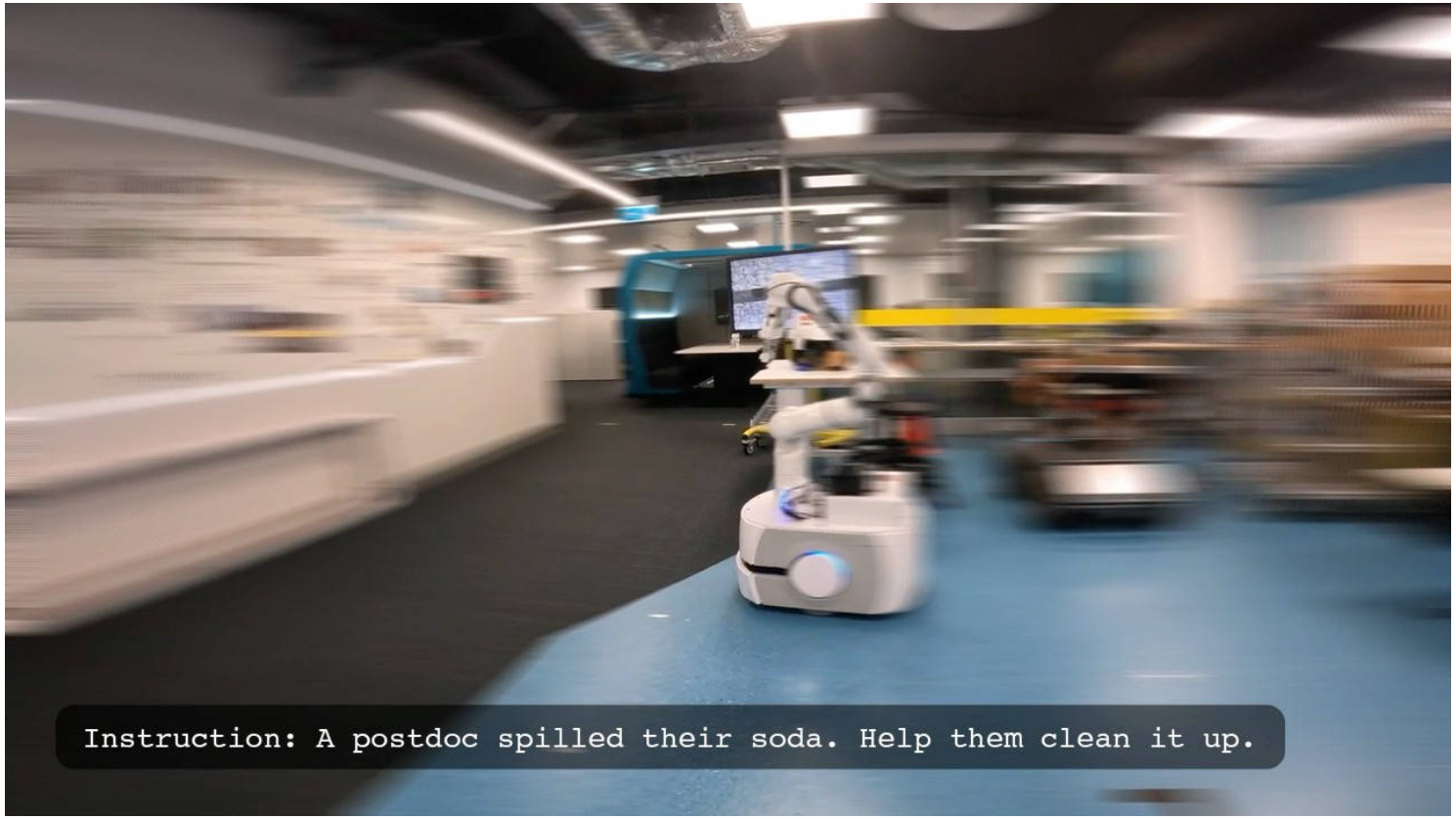
What does this robot need to …

… see?

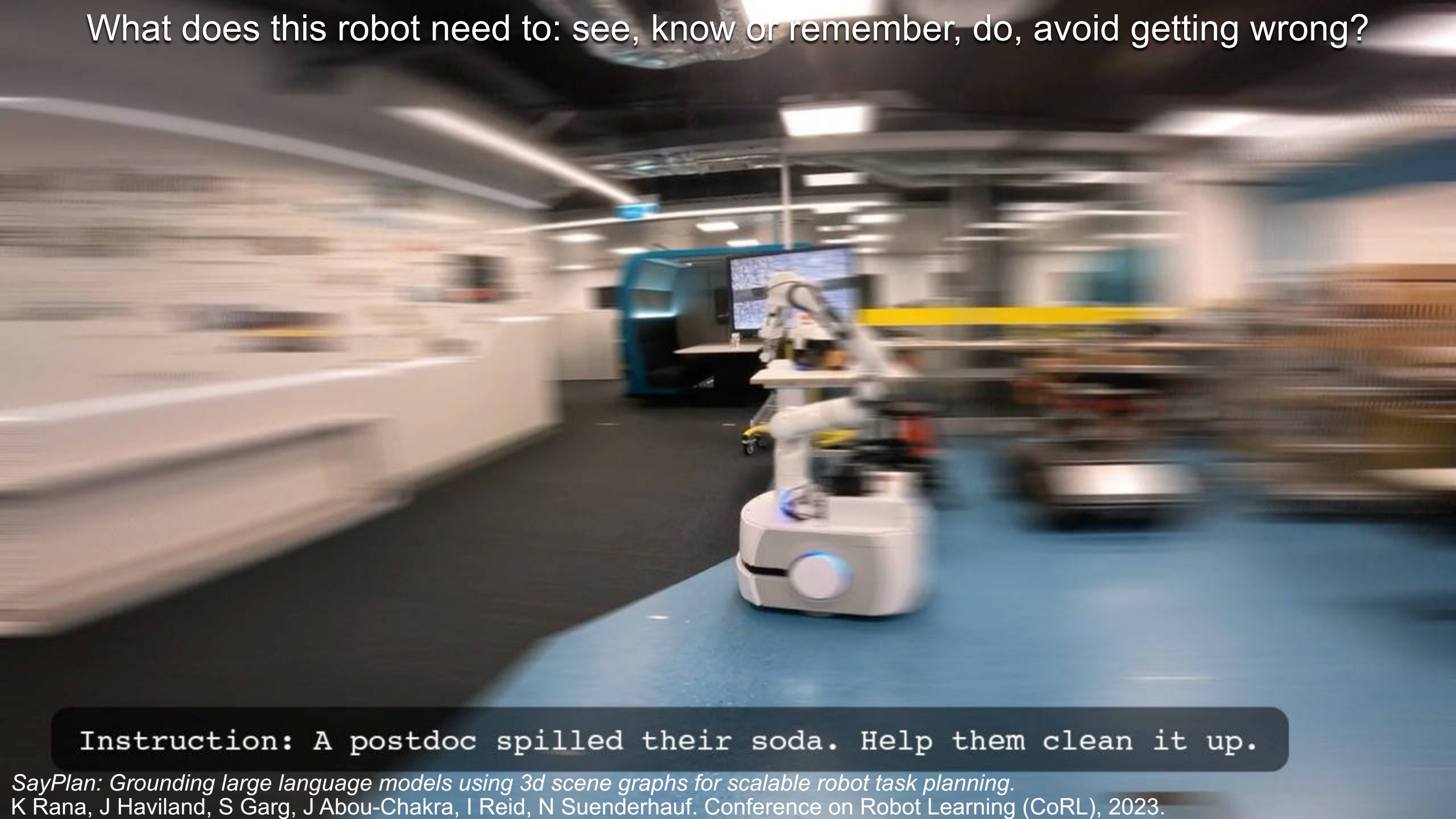
… know or remember?

… do?

… avoid getting wrong?



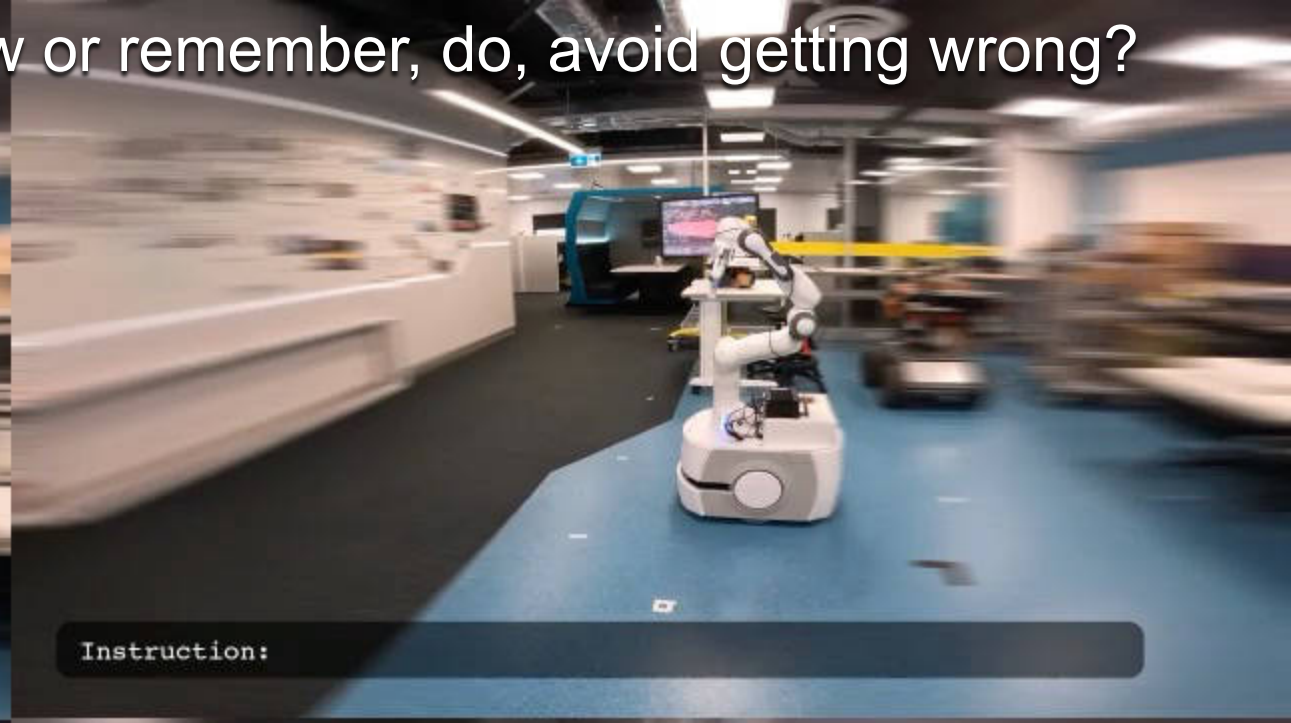
What does this robot need to: see, know or remember, do, avoid getting wrong?

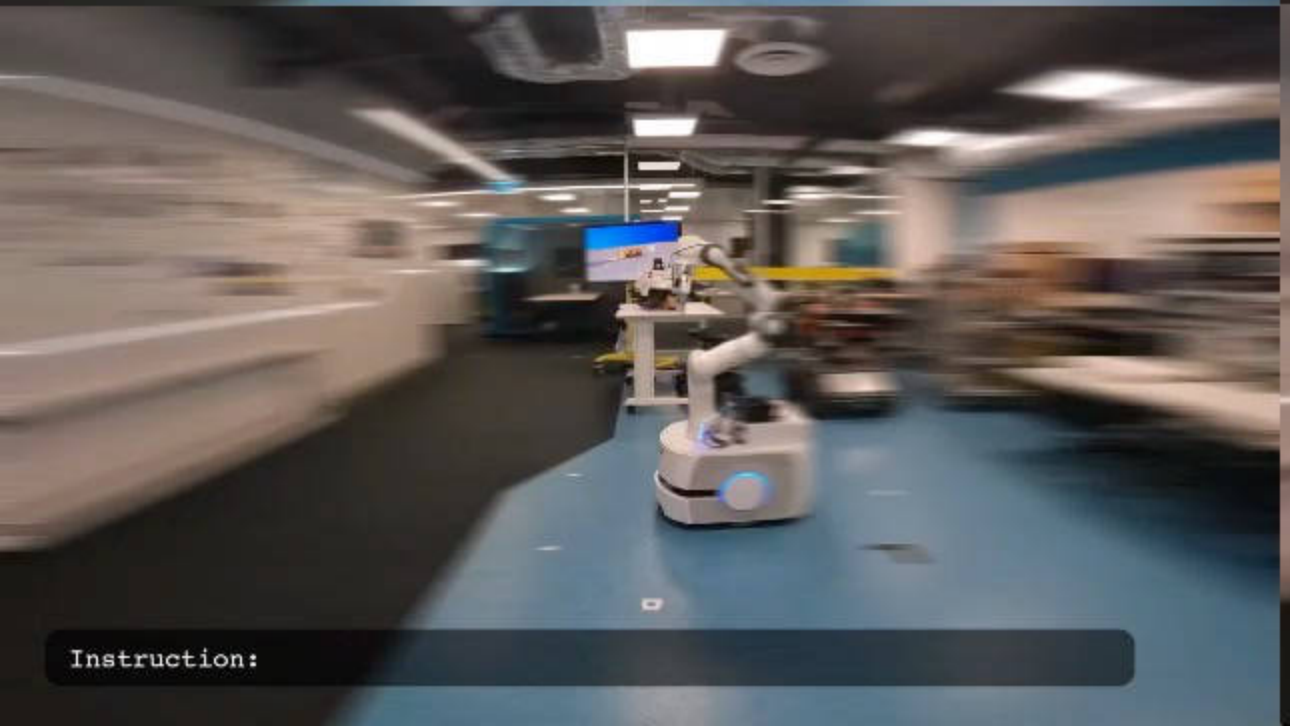


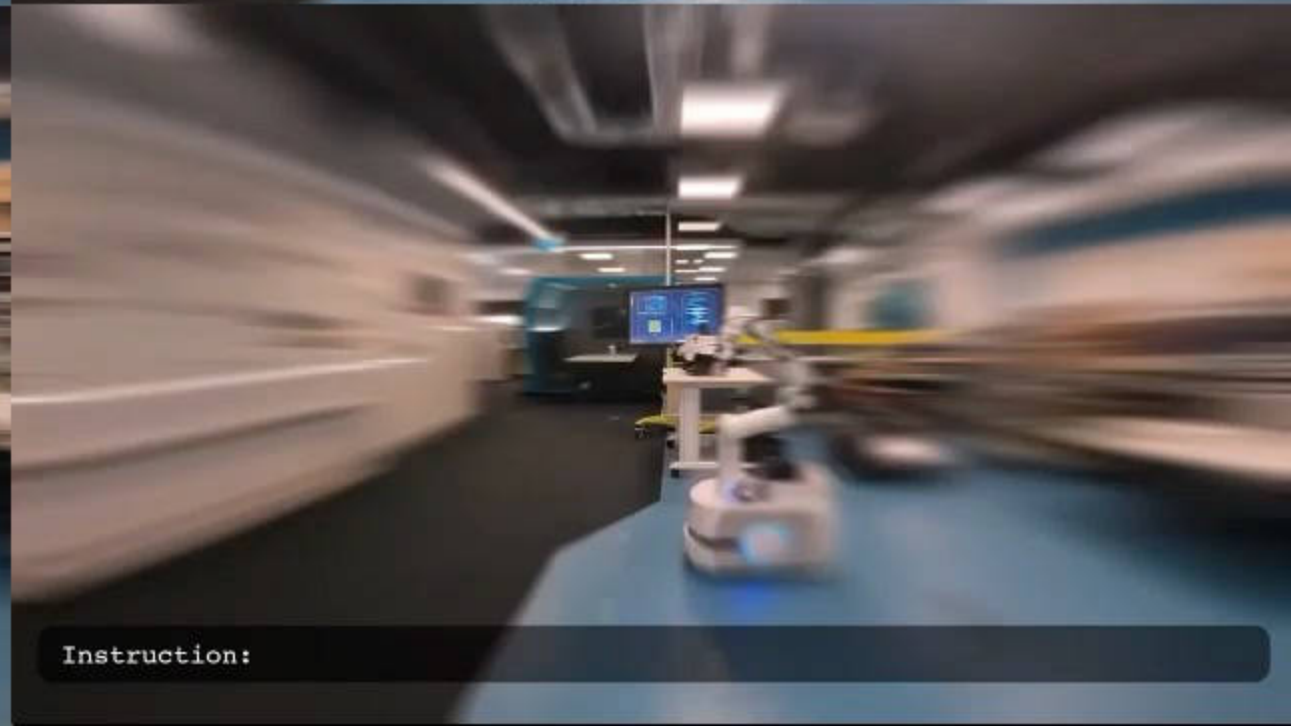
SayPlan: Grounding large language models using 3d scene graphs for scalable robot task planning.
K Rana, J Haviland, S Garg, J Abou-Chakra, I Reid, N Suenderhauf. Conference on Robot Learning (CoRL), 2023.

What does this robot need to: see, know or remember, do, avoid getting wrong?









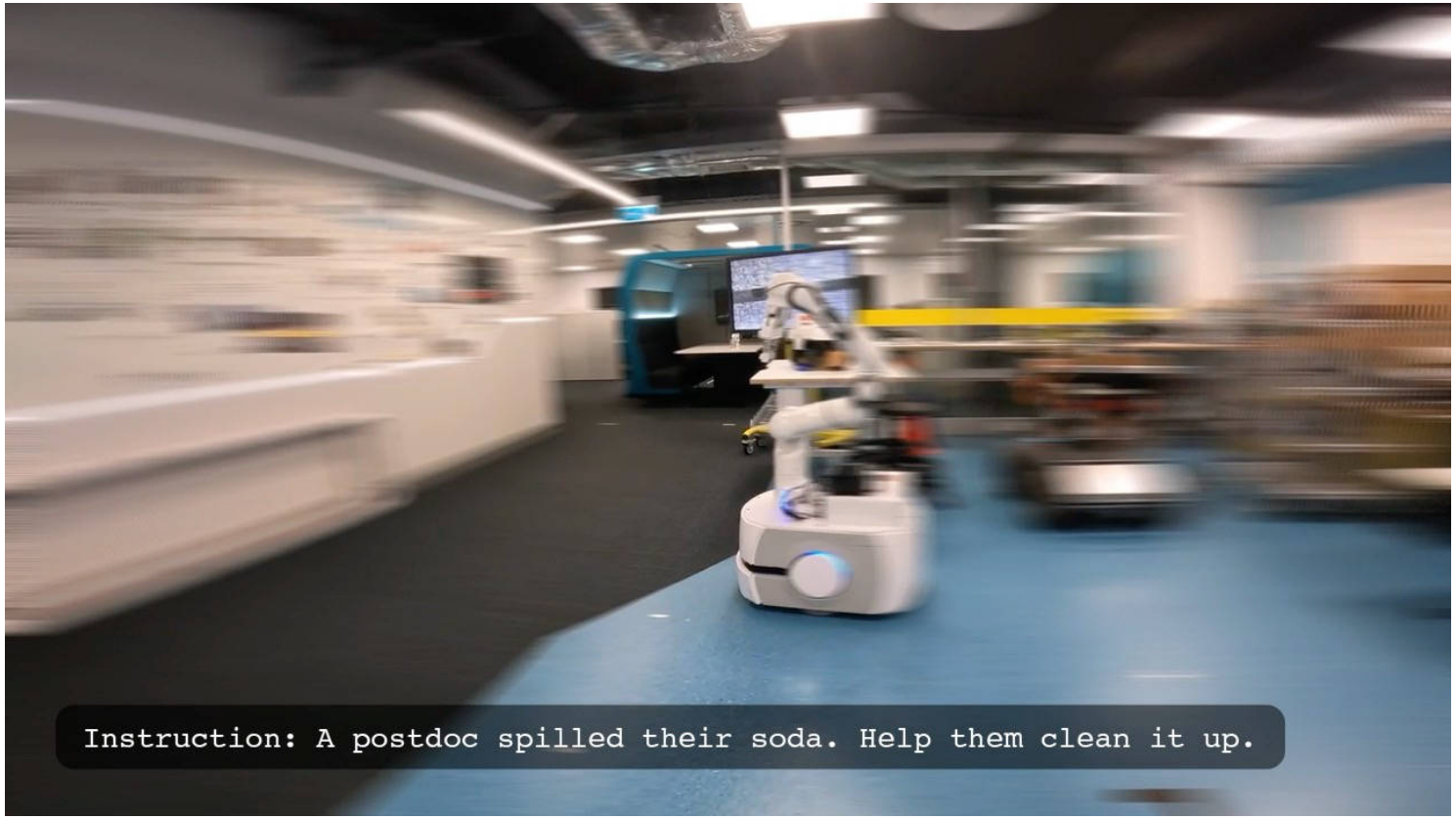
What does this robot need to …

… see?

… know or remember?

… do?

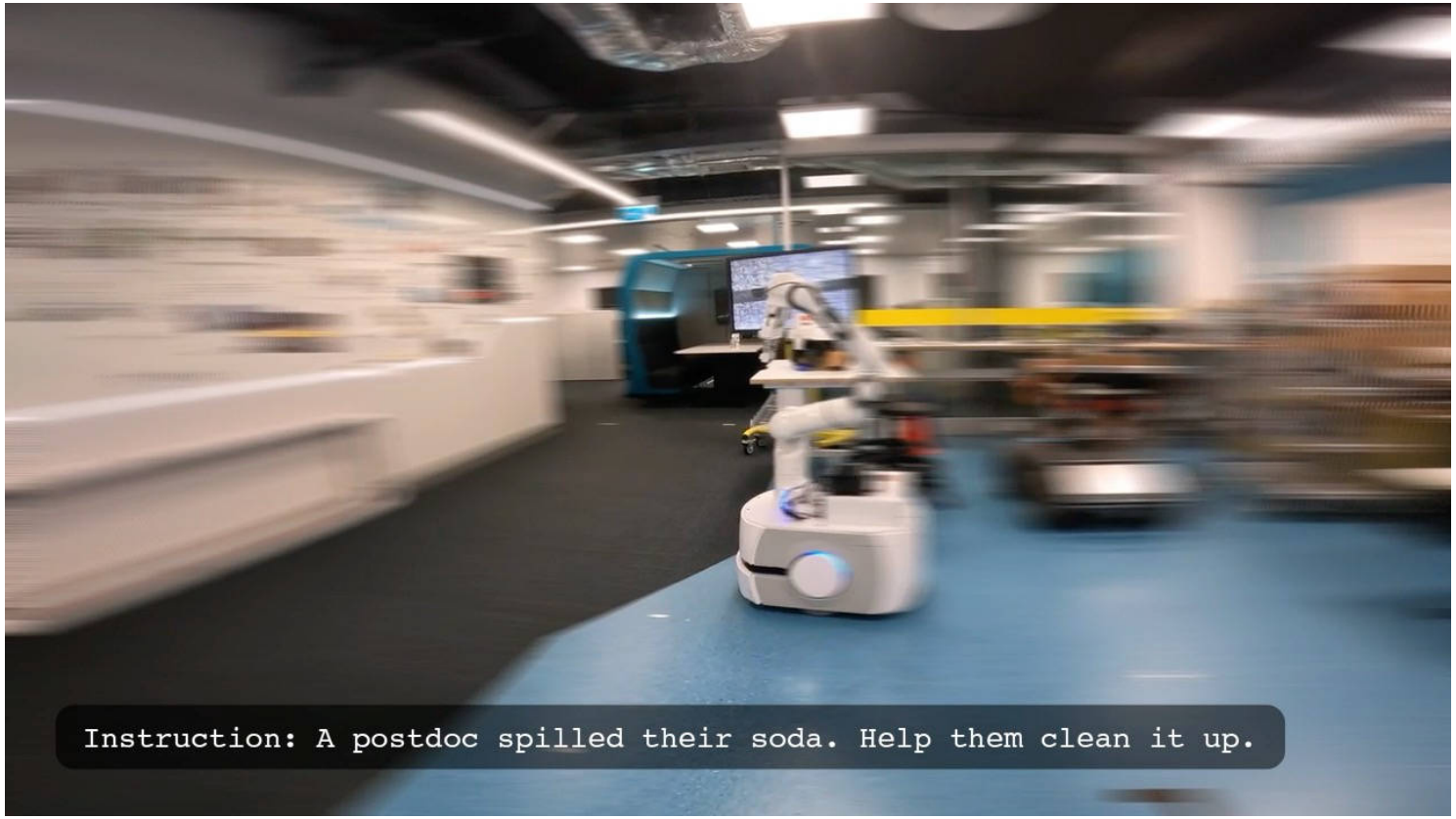
… avoid getting wrong?



The Hidden Robotic Vision Pipeline

This task requires:

- Motion Estimation
- Localisation
- Mapping
- Navigation
- Object detection
- 3d geometry
- Grasping
- Planning
- Failure awareness



CRICOS No.00213J

So, what is robotic vision?

Robotic vision is the use of cameras and algorithms to estimate information about the world that a robot needs for actions.

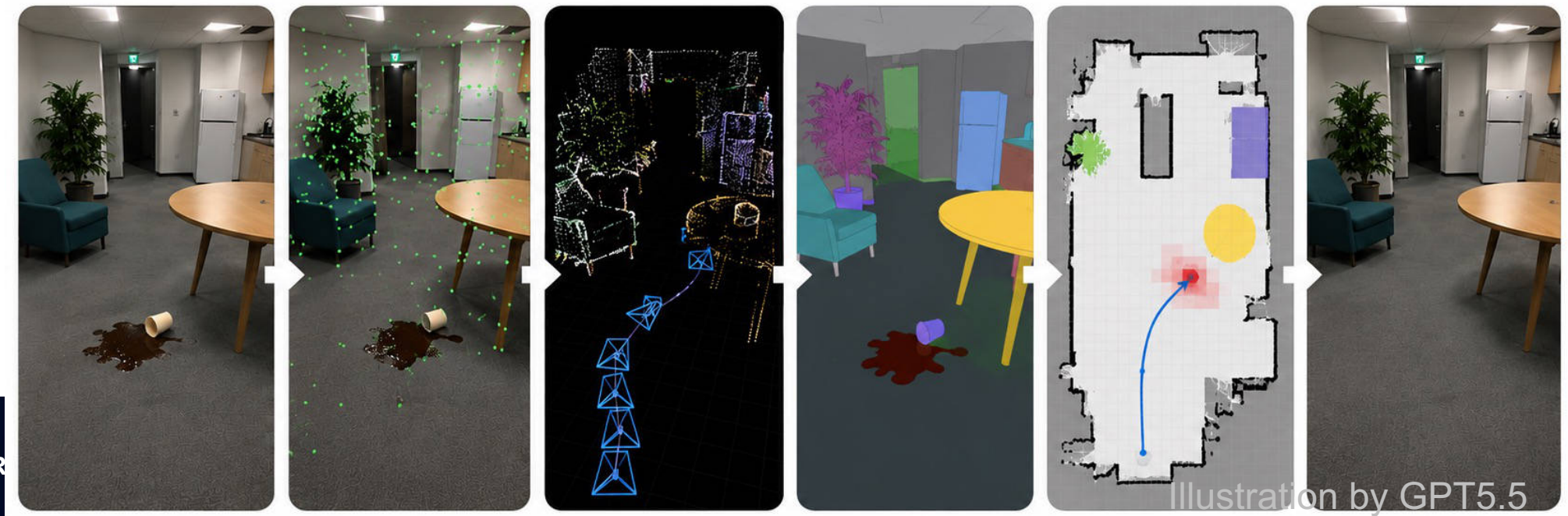


Illustration by GPT5.5

So, what is robotic vision?

Robotic vision is the use of cameras and algorithms to estimate information about the world that a robot needs for actions.

It is the integration of computer vision algorithms with robotic actuators, allowing a robot to perceive its environment and execute physical actions based on that visual data.

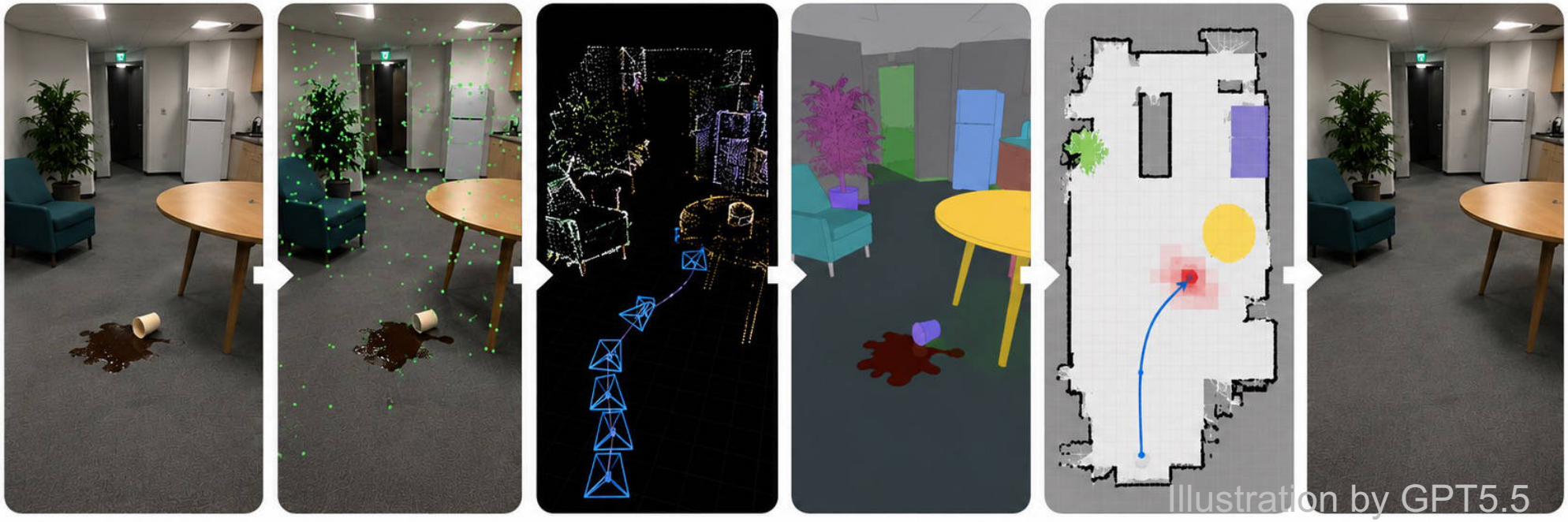


Illustration by GPT5.5

Robotic Vision vs. Other Fields

Field	Input	Output	Examples
Image Processing	Image	Image	Blurring, sharpen, “improve”
Computer Graphics	Data	Image	Rendering, game engines
Computer Vision	Image	Information	Object labels, bounding boxes, depth, pose
Robotic Vision	Image	Action-relevant information	Where to move, where to grasp
Robot Learning (ENN585 next Sem)	Image + actions + data	Learned behaviours	Reinforcement learning, Imitation learning from demonstrations

CRICOS No.00213J

The Big Questions in ENN583

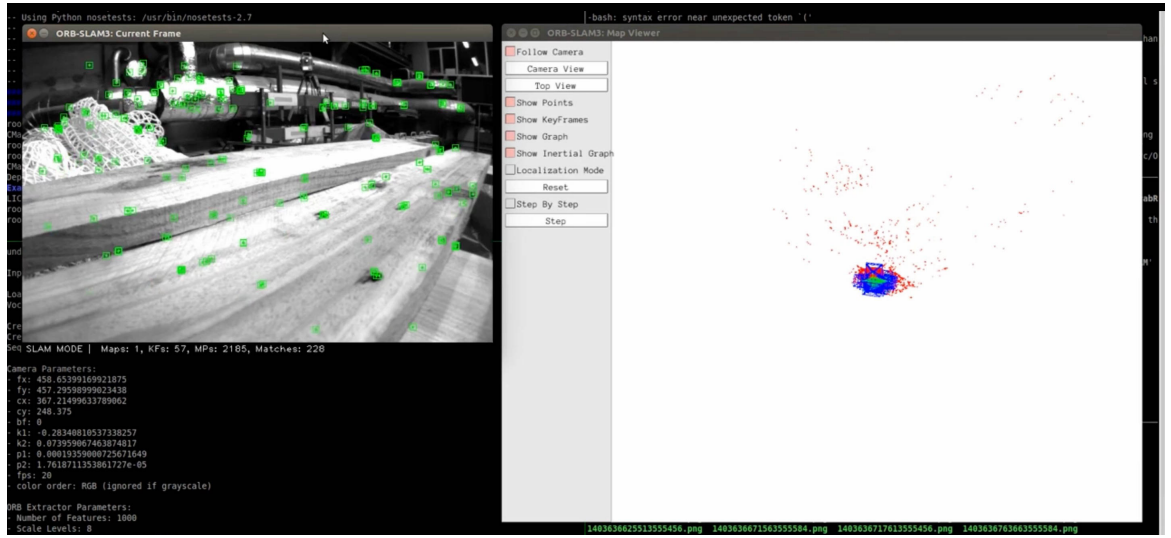
- **Where am I?**
  - Camera geometry, features, matching, visual odometry, SLAM.
- **What is around me?**
  - Objects, segmentation, scene representations, open-vocabulary perception.
- **When does vision fail?**
  - Occlusion, lighting, ambiguity, uncertainty, distribution shift, safety.

CRICOS No.00213J

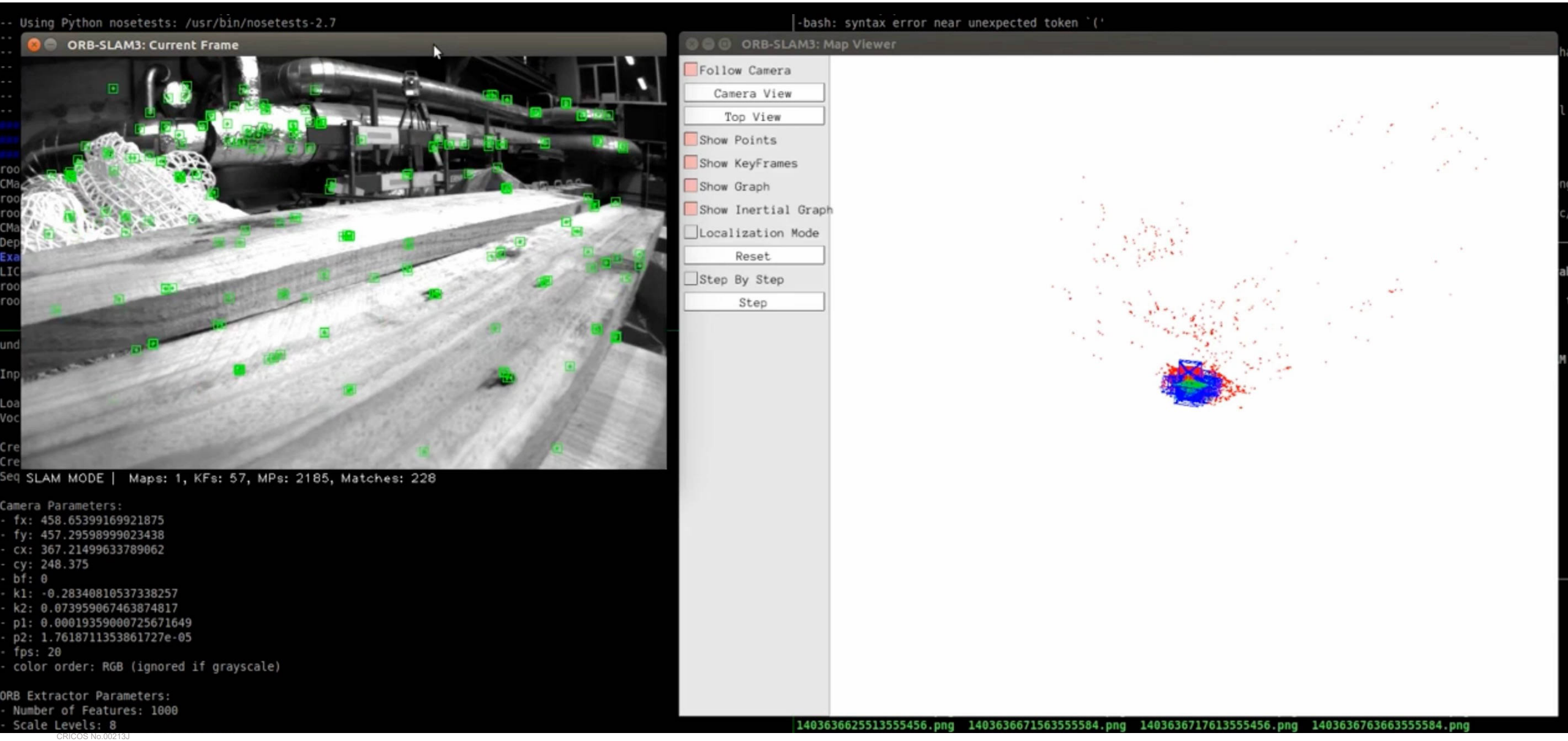
Some Examples …

Example 1 – Visual Odometry

We can estimate camera motion and 3D structure of the world by matching visual evidence across images.



ORB-SLAM3 Demo on EuRoC MH01 dataset, https://www.youtube.com/watch?v=WfY6mCTnFGg

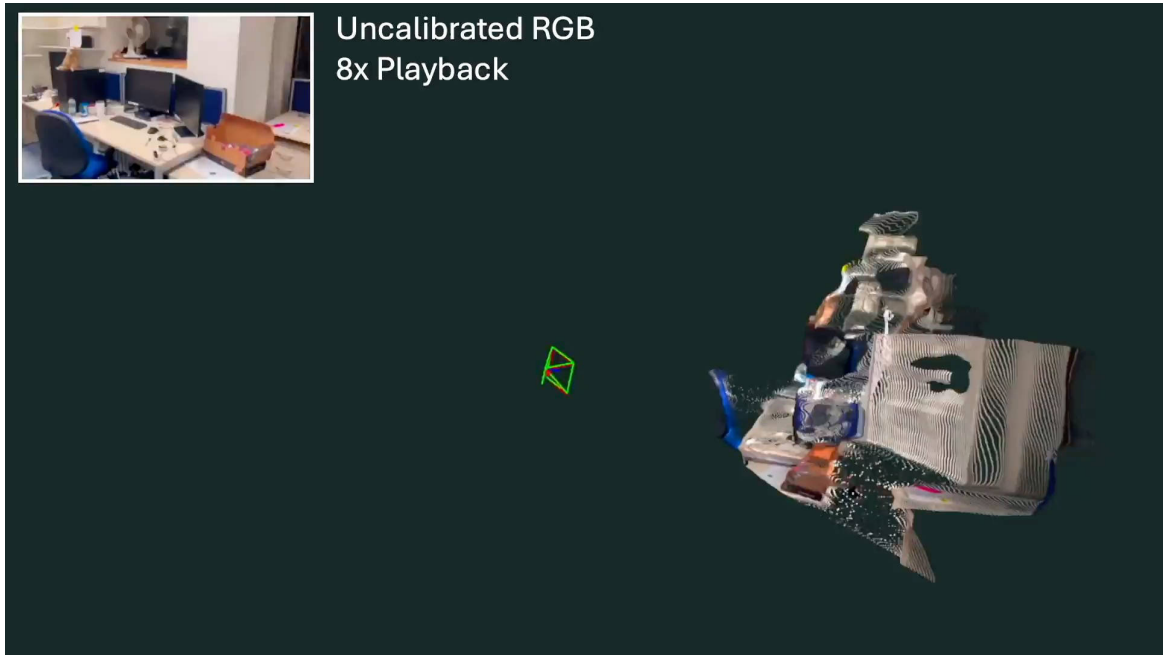


CRICOS No.00213J

ORB-SLAM3 Demo on EuRoC MH01 dataset, https://www.youtube.com/watch?v=WfY6mCTnFGg

Example 2 – Visual SLAM

A robot can build a consistent map of the world by aggregating motion information and correcting estimation errors.





Uncalibrated RGB
8x Playback

Riku Murai, Eric Dexheimer, Andrew J. Davison. MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors. CVPR 2025.

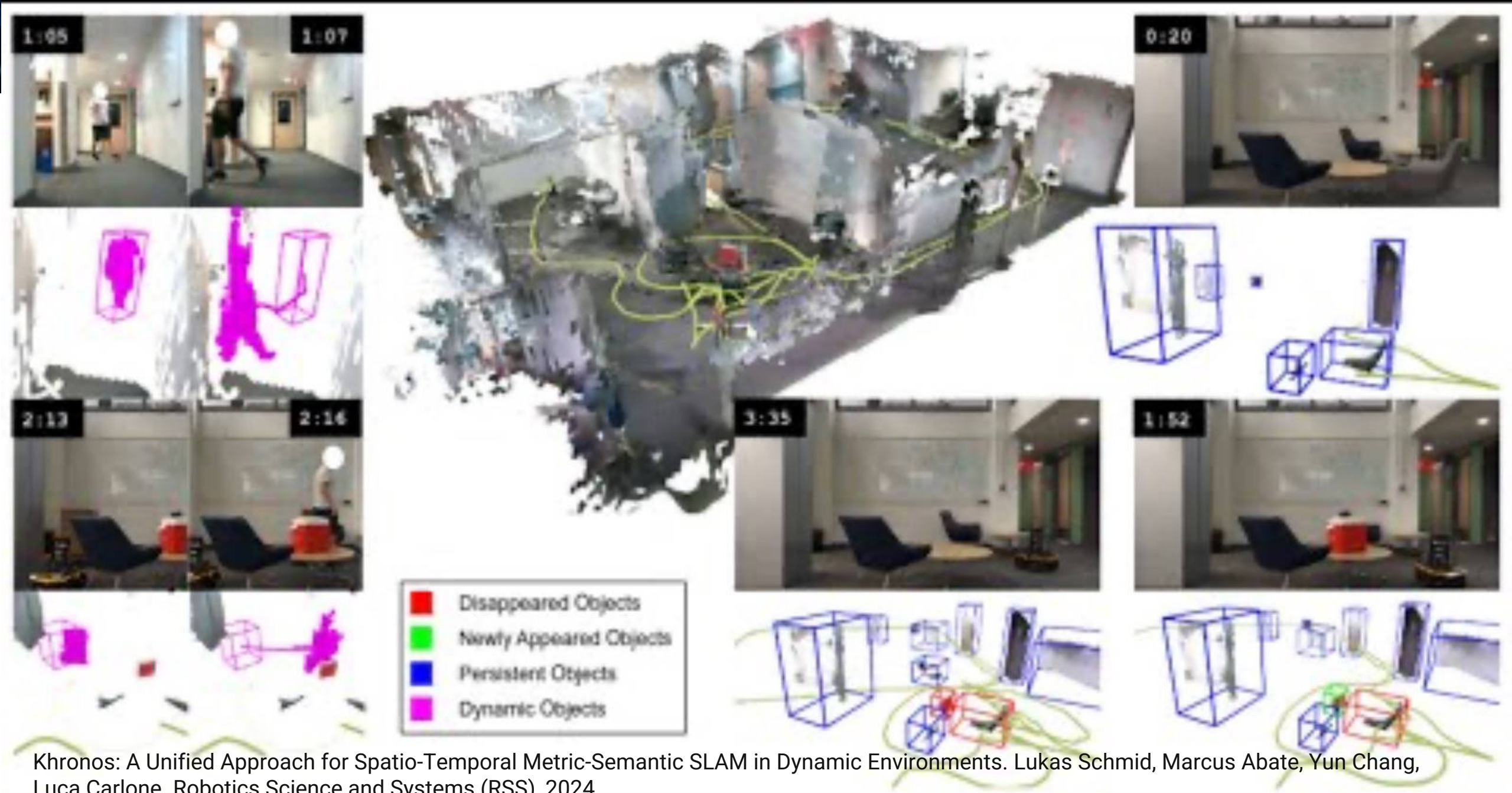
Example 3 – Scene Understanding

Robots can understand the world around them by using neural networks that are trained to detect objects and other relevant information.





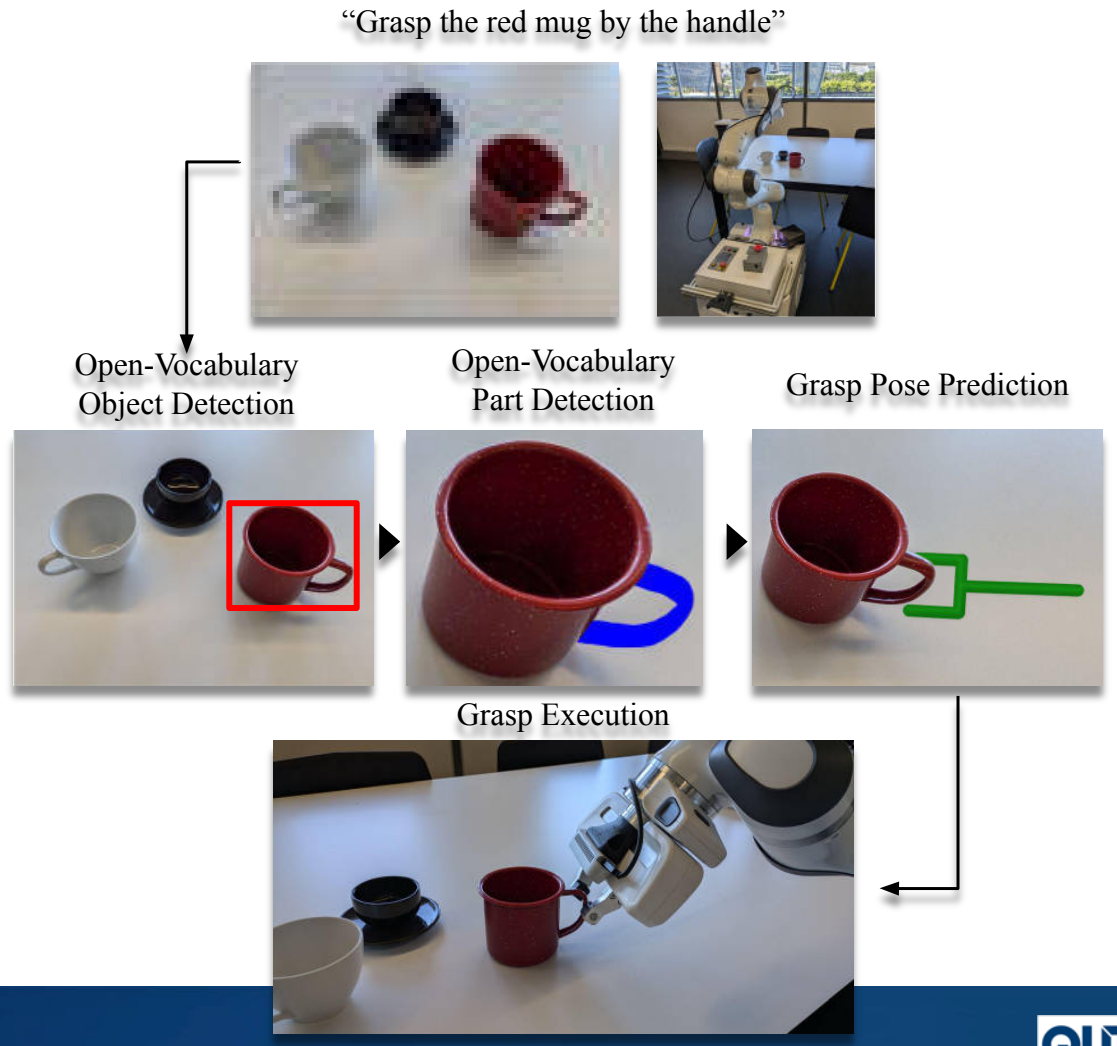

bottle|0.95
car|0.81
vase|0.935
vase|0.921
vase|0.864
vase|0.863
orange|0.88
book|0.84
clock|0.84
keyboard|0.96
mouse|0.95
bowl|0.97
teddy bear|0.98
cup|0.95
cup|0.96
spoon|0.97
ork|0.96
knife|0.76
cup|0.68
cup|0.92

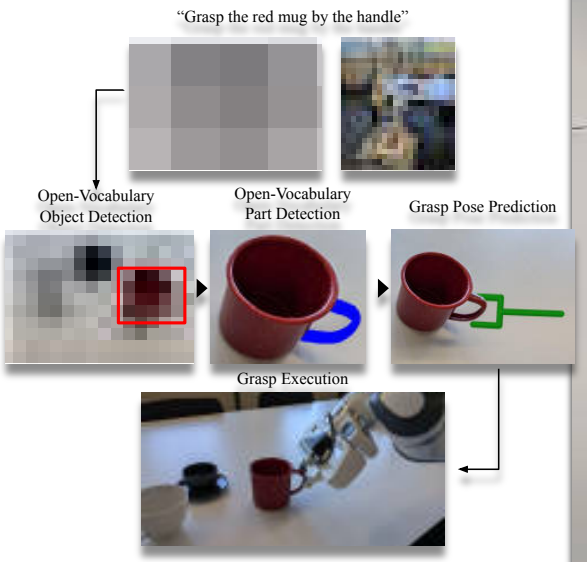


Khronos: A Unified Approach for Spatio-Temporal Metric-Semantic SLAM in Dynamic Environments. Lukas Schmid, Marcus Abate, Yun Chang, Luca Carlone. Robotics Science and Systems (RSS), 2024

Example 4 – Grasping Objects

A robot can interact in a targeted way with the world by combining language understanding, semantic perception on multiple levels, and control.





CRICOS No.00213J

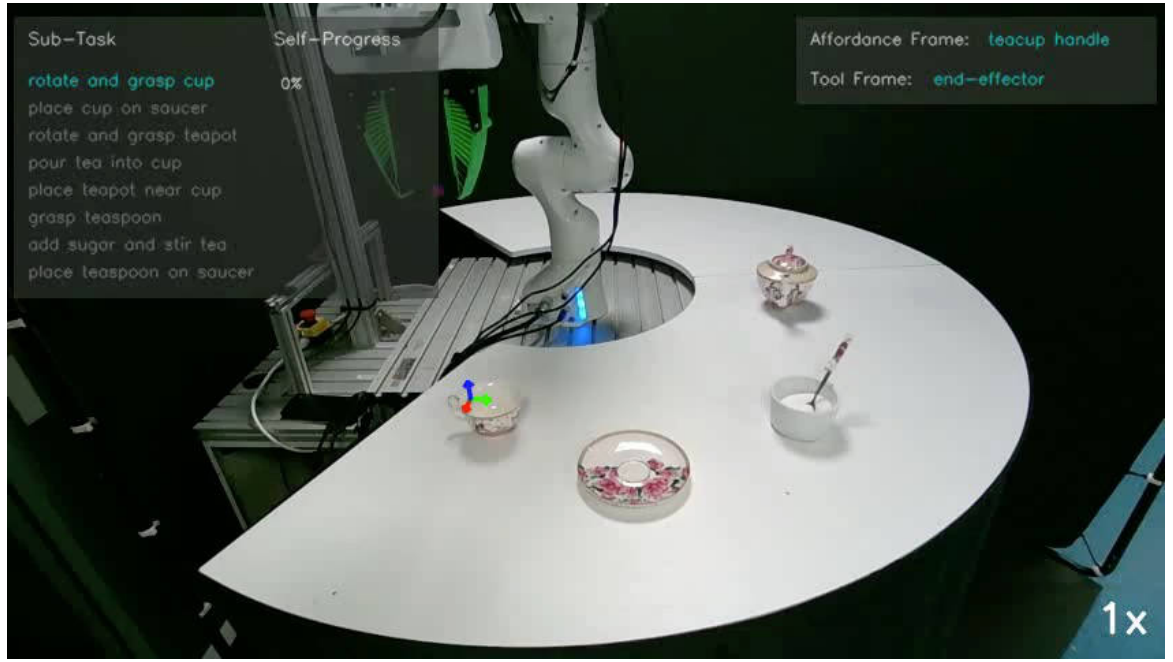
Centre for Robotics

Open-Vocabulary Part-Based Grasping.

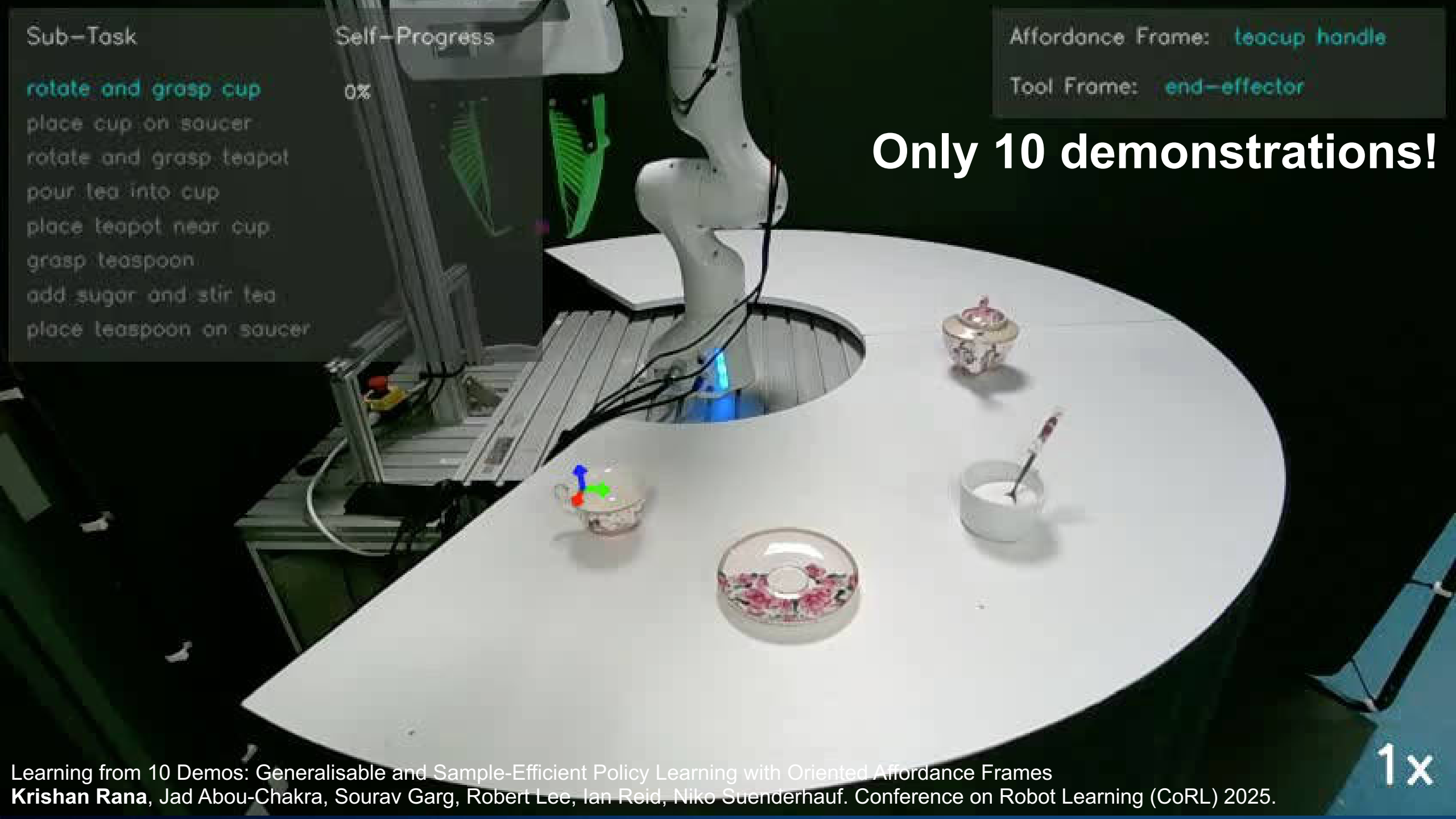
Tjeard van Oort, Dimity Miller, Will N. Browne, Nicolas Marticorena, Jesse Haviland, Niko Suenderhauf

Example 5 – Learning from Images

Robots can learn complex perception-to-action behaviours, but they still depend on the foundations: images, geometry, representations, uncertainty.



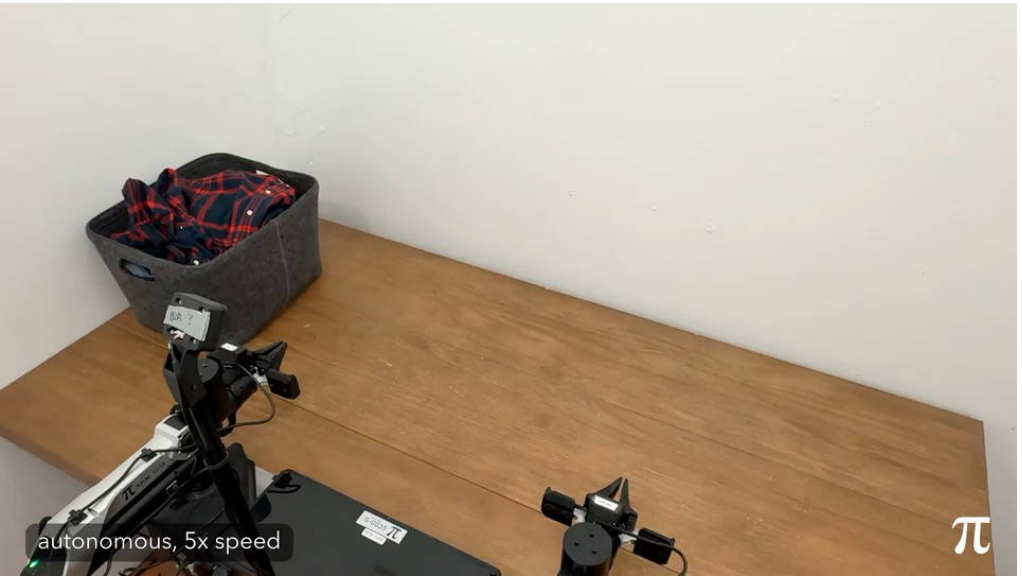
Learning from 10 Demos: Generalisable and Sample-Efficient Policy Learning with Oriented Affordance Frames
Krishan Rana, Jad Abou-Chakra, Sourav Garg, Robert Lee, Ian Reid, Niko Suenderhauf. Conference on Robot Learning (CoRL) 2025.



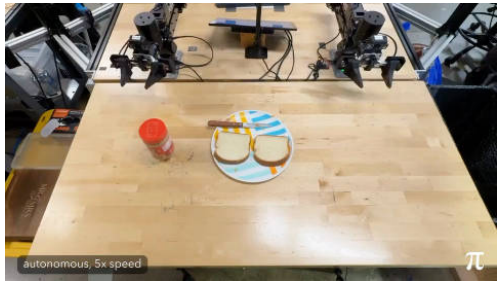
Learning from 10 Demos: Generalisable and Sample-Efficient Policy Learning with Oriented Affordance Frames
Krishan Rana, Jad Abou-Chakra, Sourav Garg, Robert Lee, Ian Reid, Niko Suenderhauf. Conference on Robot Learning (CoRL) 2025.

Robot Learning is getting really good!

Video Credits: π0.7: a Steerable Model with Emergent Capabilities. https://www.pi.website/blog/pi07

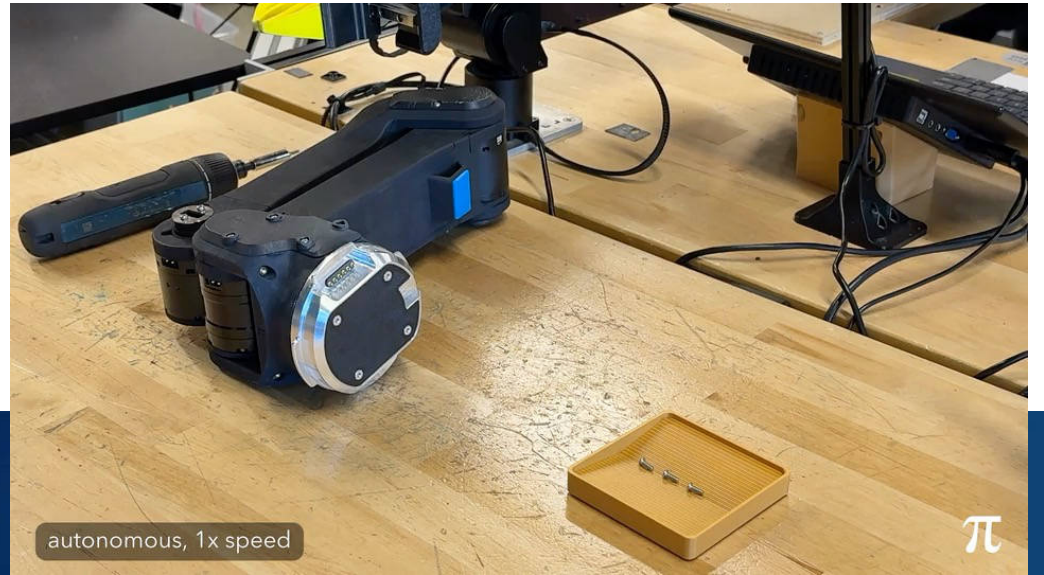






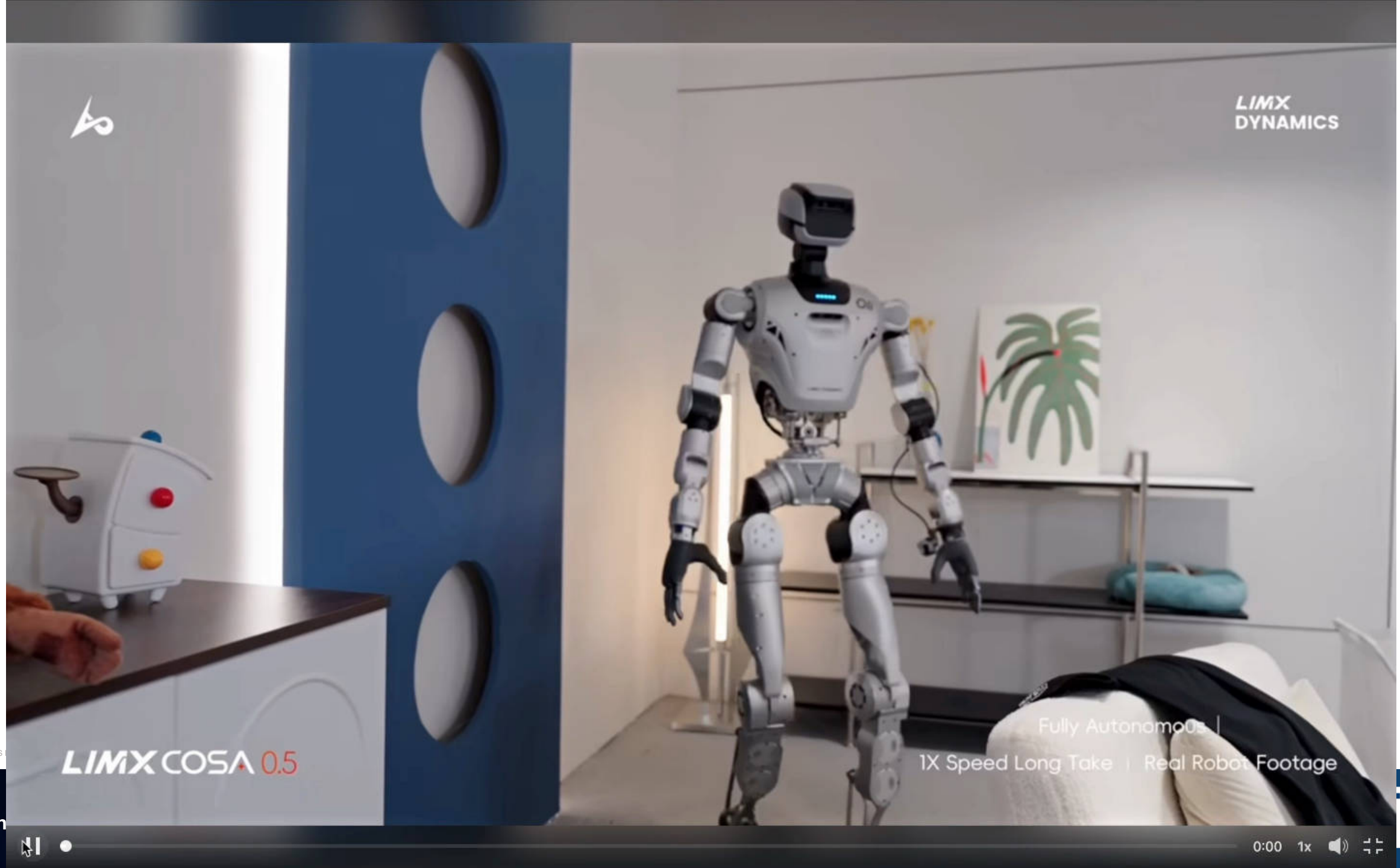











LIMX DYNAMICS
LIMX COSA 0.5
Fully Autonomous |
1X Speed Long Take | Real Robot Footage
0:00
1x

What do all these examples have in common?

What do these examples have in common?

They all start with images, rather than direct access to the true state of the world

What useful information is missing from the raw image in each example?

(visual odometry, visual SLAM, scene understanding, grasping, learning complex behaviours)

CRICOS No.00213J

What do these examples have in common?

They all start with images, rather than direct access to the true state of the world

- The image does not directly contain:
  - the camera motion;
  - the robot's location;
  - the 3D structure of the scene;
  - the identity or function of objects;
  - the correct grasp;
  - the correct action.
- Those quantities must be **inferred**.

CRICOS No.00213J

What do these examples have in common?

They extract a task-relevant representation from the images

Example	Representation produced
Visual odometry	Camera motion and trajectory
Visual SLAM	Robot trajectory plus a consistent map
Scene understanding	Object bounding boxes or segments
Object grasping	Object pose, parts, affordances or grasp locations
Learning from images	A learned action policy

CRICOS No.00213J

What do these examples have in common?

They must deal with ambiguity and uncertainty

What could make each of these systems confidently produce wrong results?

(visual odometry, visual SLAM, scene understanding, grasping, learning complex behaviours)

CRICOS No.00213J

What do these examples have in common?

They must deal with ambiguity and uncertainty

Example	Failure case
Visual odometry	Motion in the image could come from a moving object
Visual SLAM	Two places may look very similar
Scene understanding	Novel objects might be mistaken as one of the known objects
Object grasping	Several grasps might be possible, objects might be occluded
Learning from images	The learned action policy might encounter a situation that is very different to its training data

Semester Roadmap

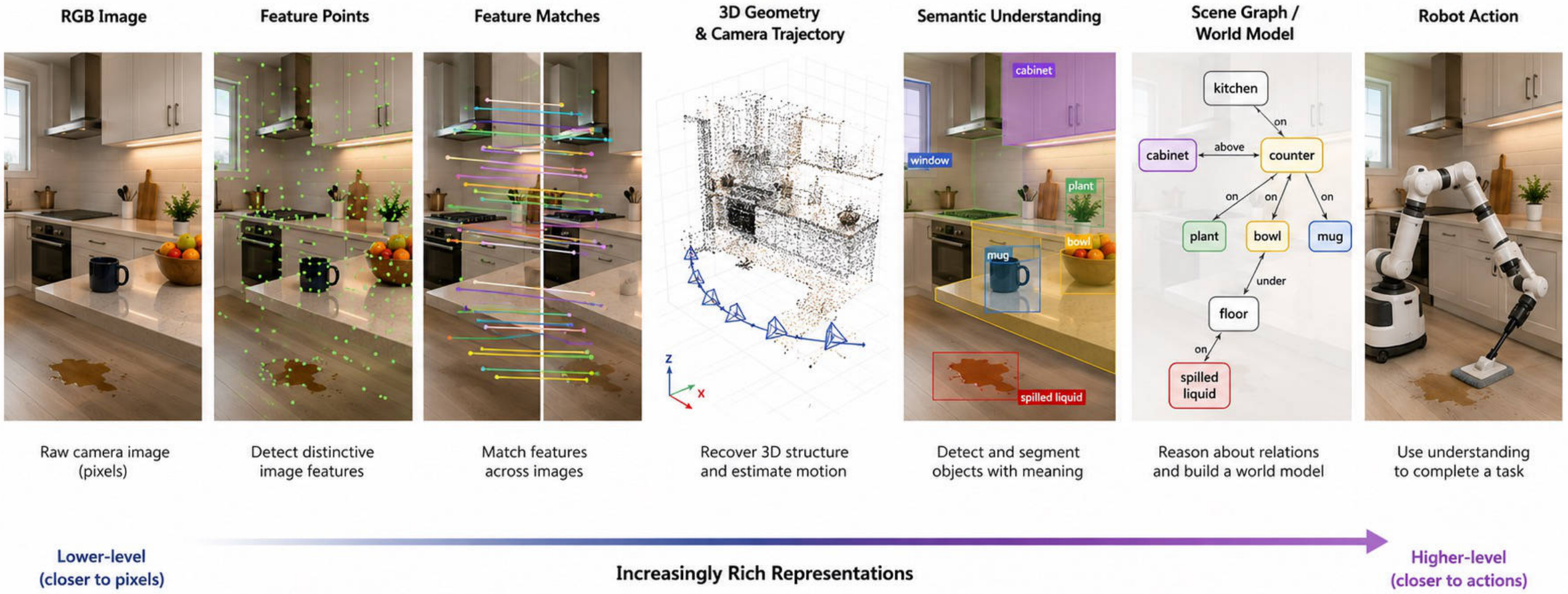


Illustration by GPT5.5

Semester Roadmap

- Weeks 1-7: Geometry-based methods
  - Features & Matching
  - Cameras, Stereo, Multi-View
  - Visual Odometry
  - SLAM
- Weeks 8-13: Learning-based methods
  - Deep learning
  - Representations
  - Detection, Segmentation, Retrieval
- Throughout: Limitations and failure modes

CRICOS No.00213J

The most important thing: Ideas, not details



Don’t get lost in the details.

For every topic we study, ask:

What is the underlying idea?

CRICOS No.00213J

The most important thing: Ideas, not details



Don’t get lost in the details.

For every topic we study, ask:

What is the underlying idea?

Then look at the big-picture context:

- What problem is this trying to solve?
- What information is being estimated?
- Why does a robot need that information?
- What assumptions make it work?
- When will it fail?

CRICOS No.00213J

Don’t be a one-tool engineer.



- Deep learning is a powerful tool.
- Non-learning methods are also powerful tools.
  - E.g. geometry, optimisation, robust estimation, probabilistic methods, state estimation,…
- Good engineers know which tool the problem needs.

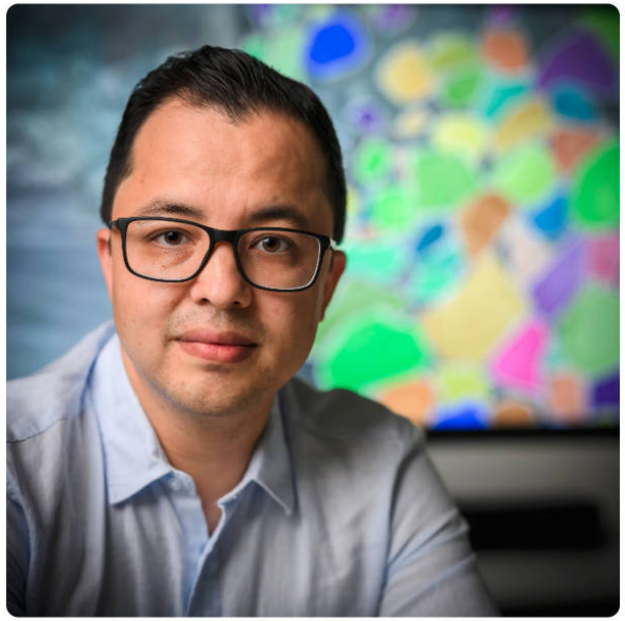
- **Use this unit** (and all the other units in the degree) **to fill your toolbox, and learn how to use it.**

CRICOS No.00213J

Your Teaching Team this Semester



Prof Niko Suenderhauf
Unit Coordinator
Lecturer



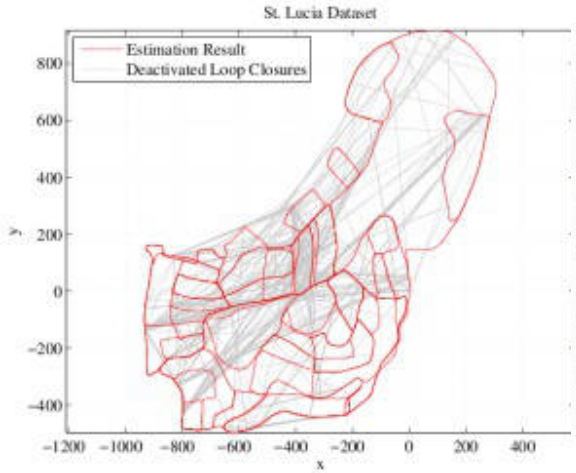
Dr Osman Tursun
Tutor (Tutorials)

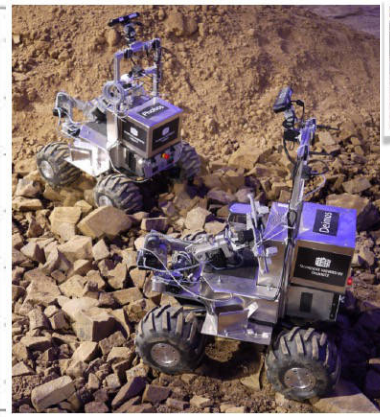


Jake Bessen
Tutor (Practicals)

My journey through Robotic Vision

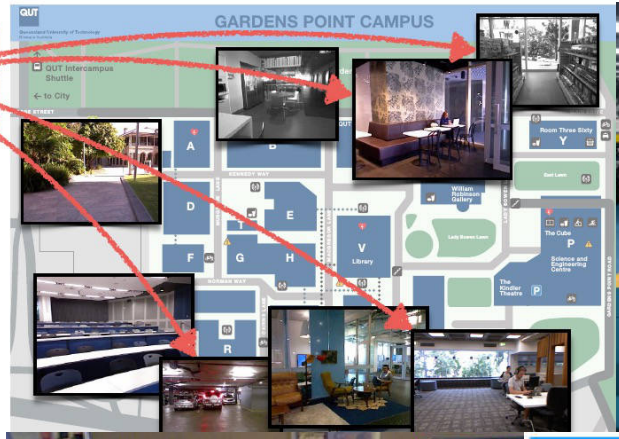
Guiding questions: Where am I? What is around me? What can I do?



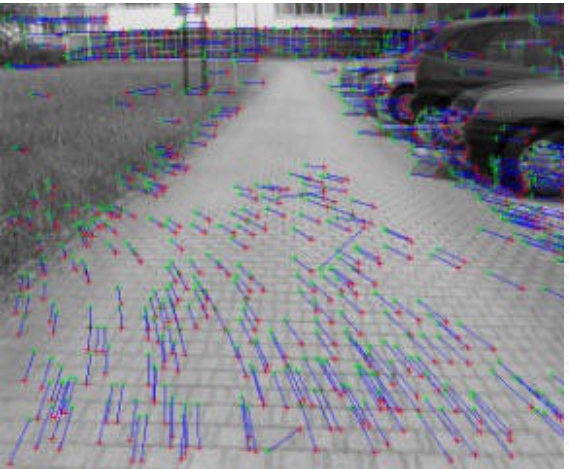


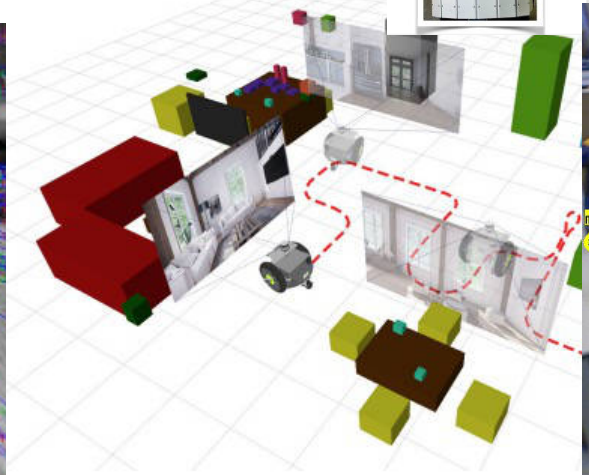


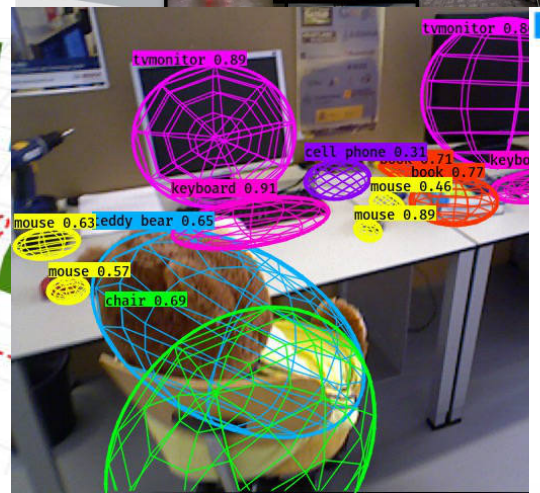


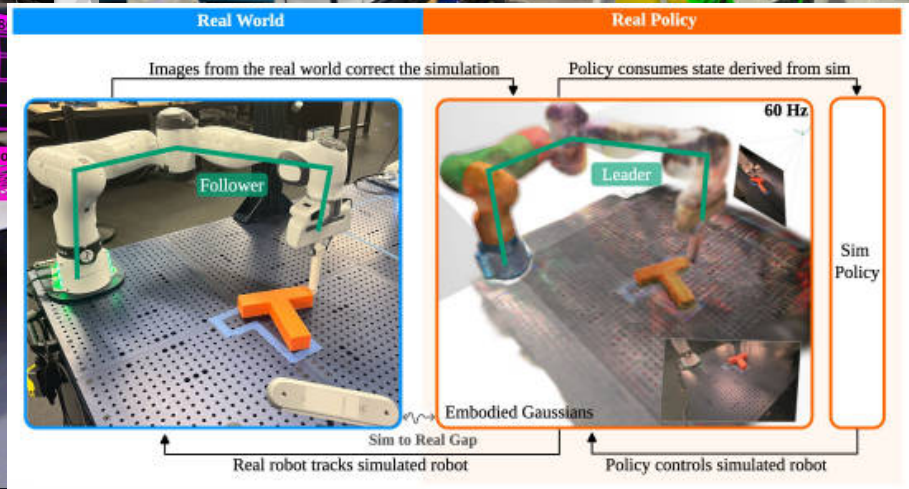








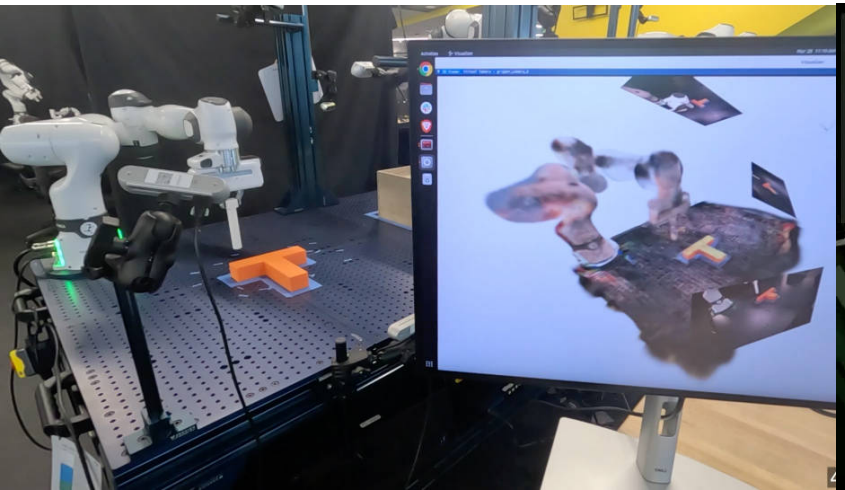


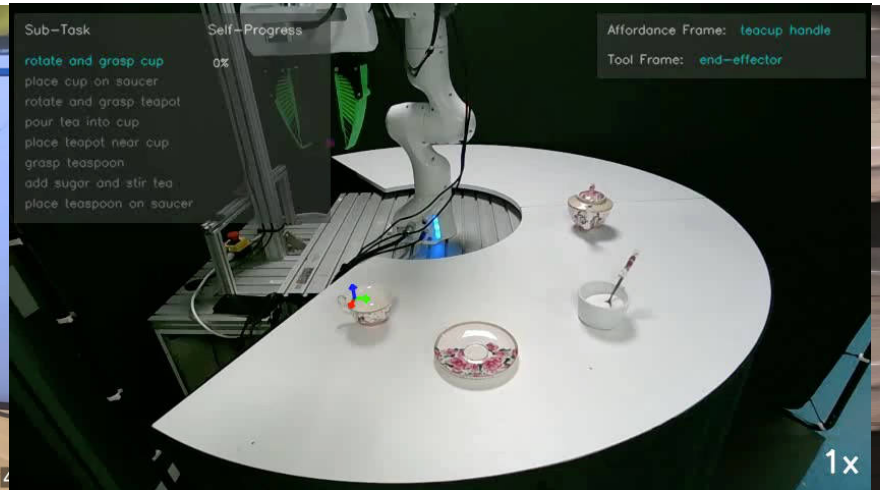


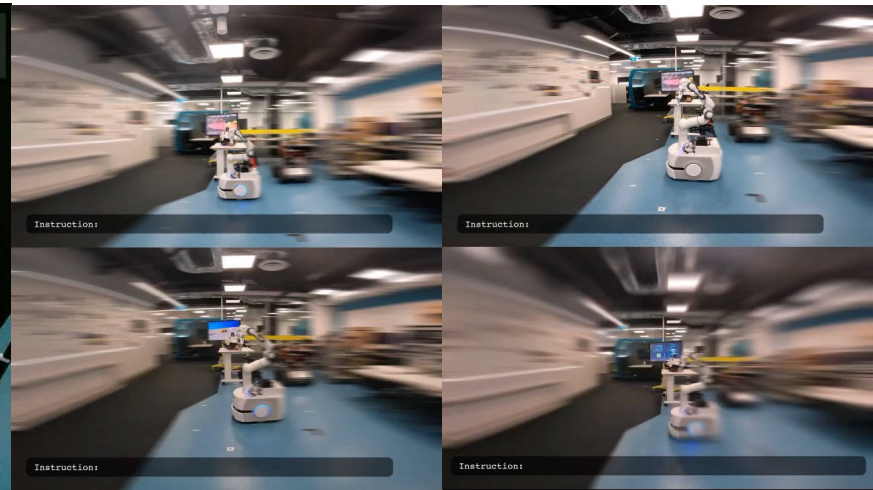
Visual Learning and Understanding

How can a robot **learn** to:

- **reliably** perceive and interpret its environment,
- build an internal **representation**
- in order to decide on its **actions**.

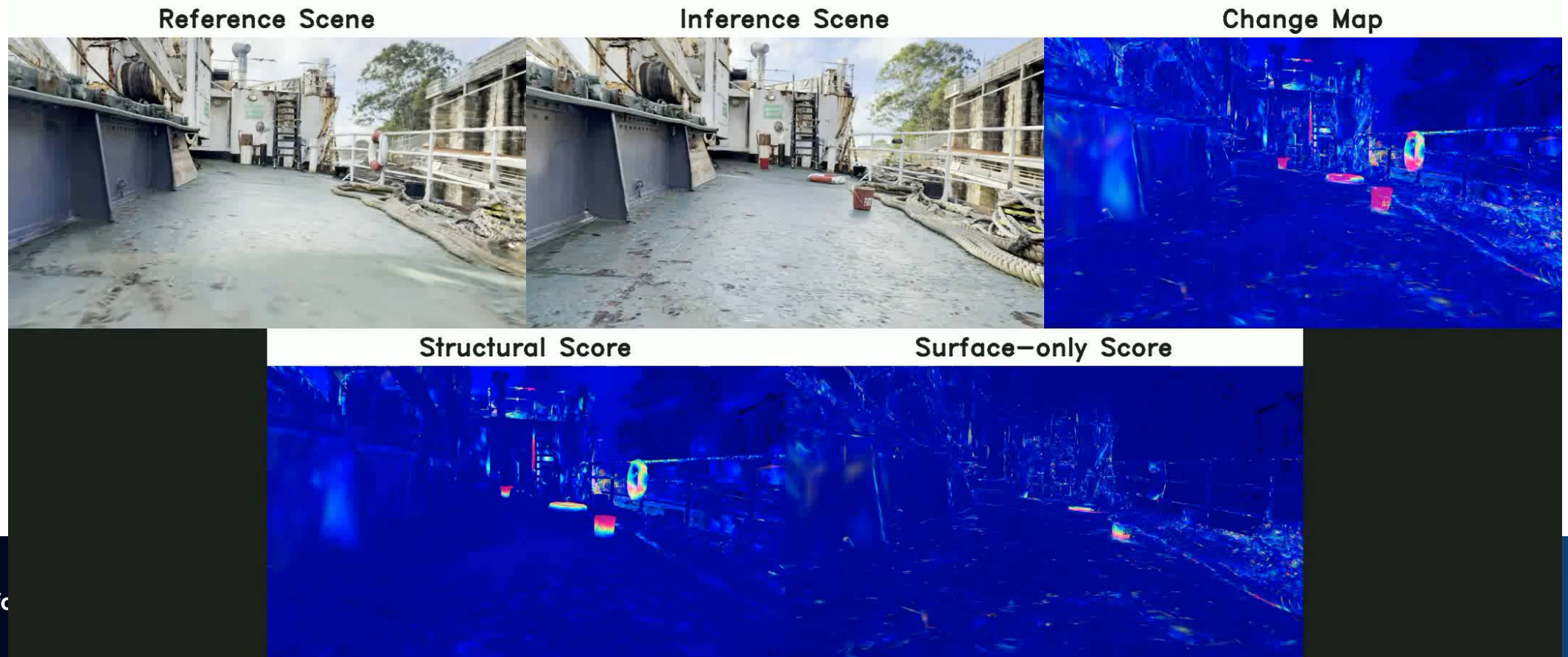






Join the team for your ENN595 project!

- Spot the Difference in 3D: Detecting Changes with 3D Gaussian Splatting
  - *ENN595: Adapt our change detection method from industrial infrastructure management to ecology (e.g. coral reefs, vegetation) or road scenarios?*



CRICOS No.00213J

How you will learn in ENN583

1. **Lecture** + some self-guided learning (2h, Mondays, 9-11am)
    - Build intuitions, introduce and connect ideas, understand the theory

2. **Tutorial** (1h, Fridays): 9-10am & 10-11am
    - Read, understand, discuss state-of-the-art papers

3. **Practical** (2h, Fridays): 12-2pm & 2-4pm
    - Implement ideas, experiment, build more intuition, learning-by-doing
    - Building blocks for your Project assessmenrt

CRICOS No.00213J

Let’s make the lectures interactive!

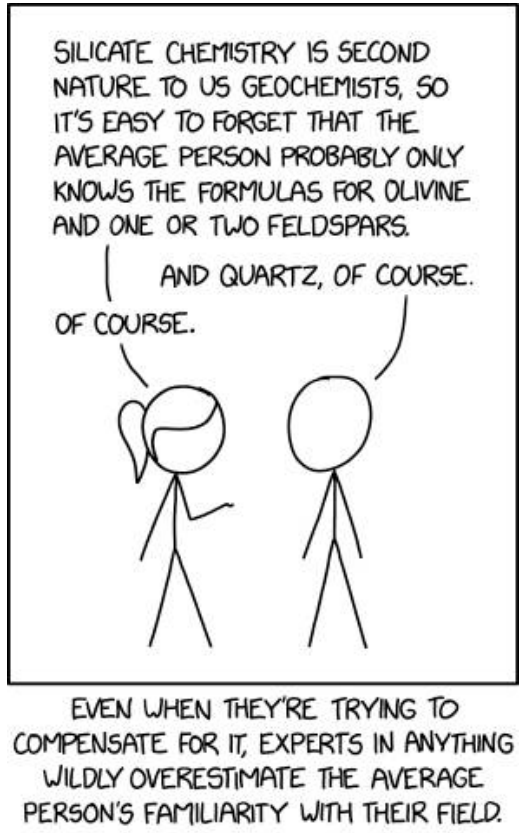
Please ask questions during the lecture!

I am more than happy to clarify, explain differently, explain again, explore connection to another idea, …

It is more rewarding and fun for you and me.

And if I don’t know the answer, I’ll look it up and explain it next week.

The curse of knowledge (by xkcd)



Assessments

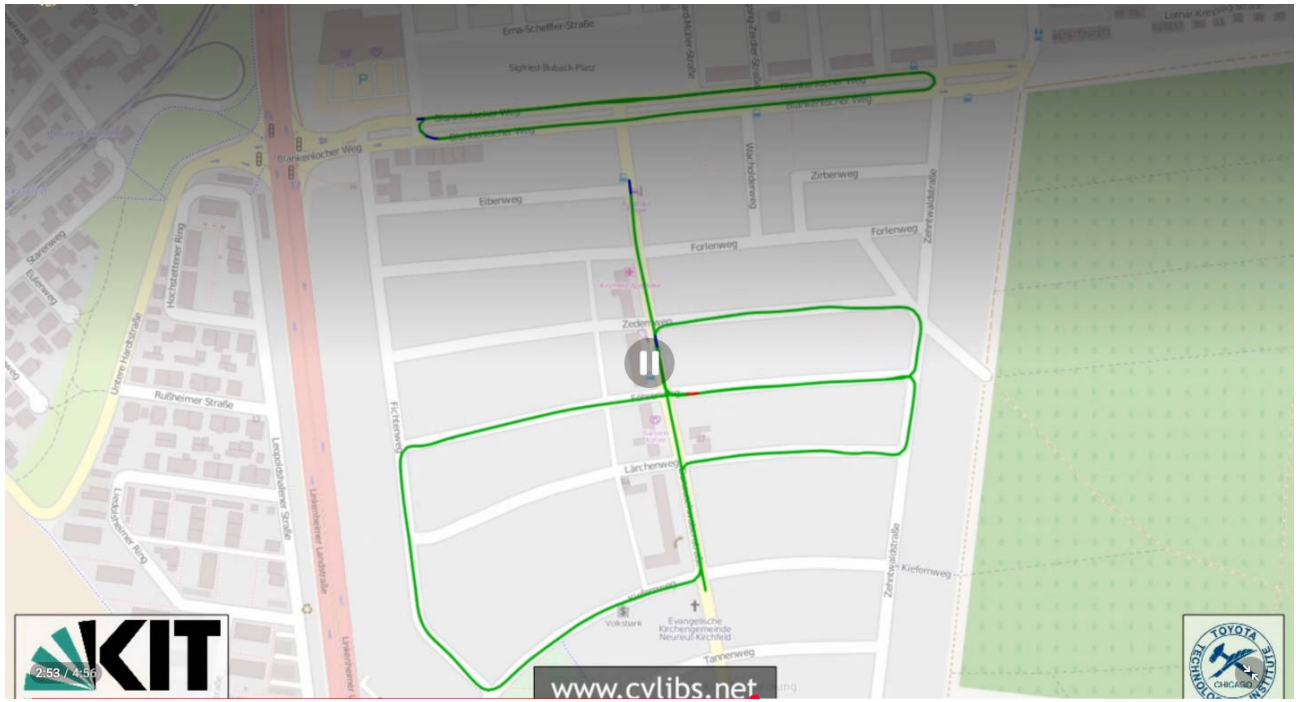
- **Applied Project**
  - Worth 40%, due Week 10 (just after mid-sem break)
  - Focus on visual odometry and related techniques
  - Code & Report

- **Exam**
  - Worth 60%
  - Focus on all unit content

CRICOS No.00213J

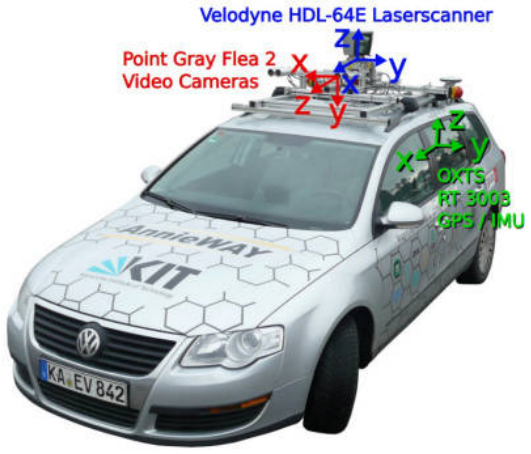
Project: What you'll build and demonstrate

From images to a robot trajectory



The project develops in three stages:

- Feature extraction and matching
- Frame-to-frame motion estimation
- Whole-trajectory motion estimation



Project: What you'll build and demonstrate

From images to a robot trajectory

The project develops in three stages:

- Feature extraction and matching
- Frame-to-frame motion estimation
- Whole-trajectory motion estimation

Submission:

- Your code
  - Automatically evaluated and graded via Gradescope
  - 20% of unit marks
- A short report
  - 20% of unit marks

CRICOS No.00213J



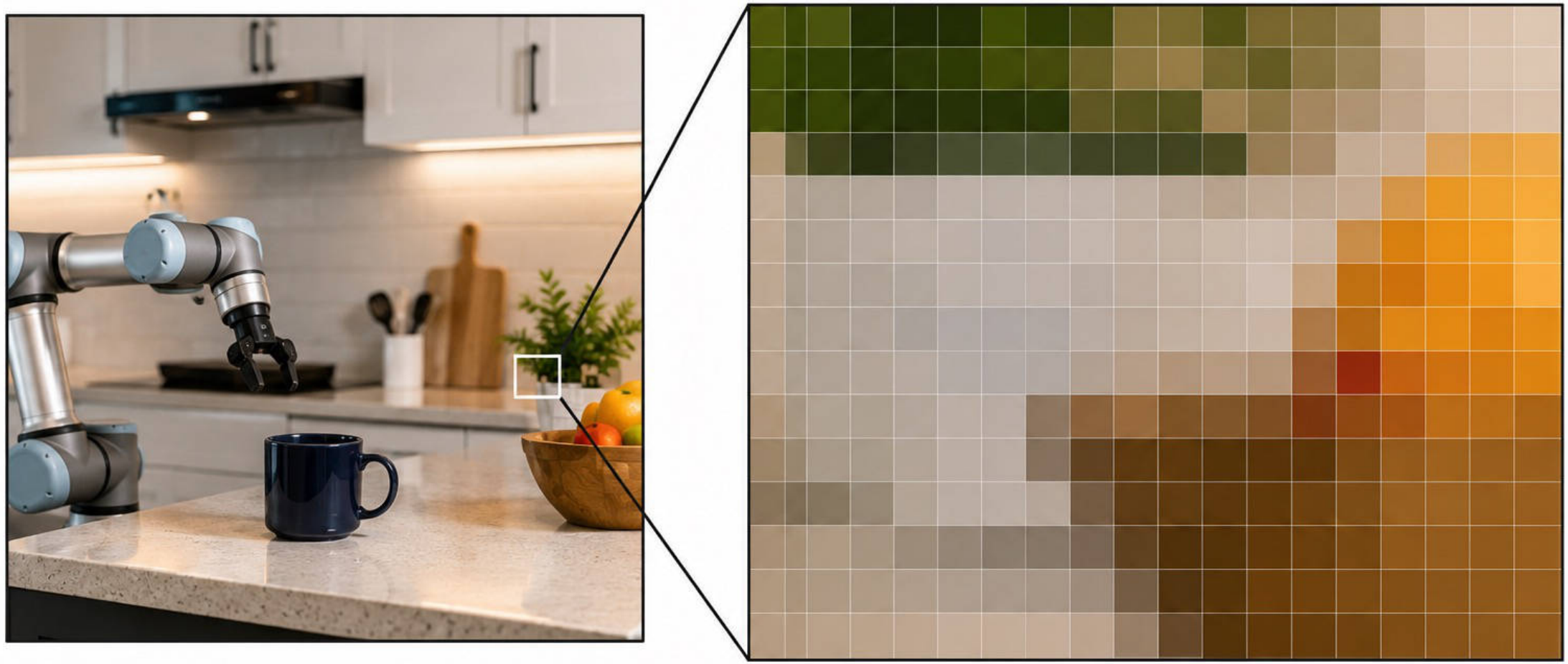
Introduction to Robotic Vision

Why is Robotic / Computer Vision Hard?

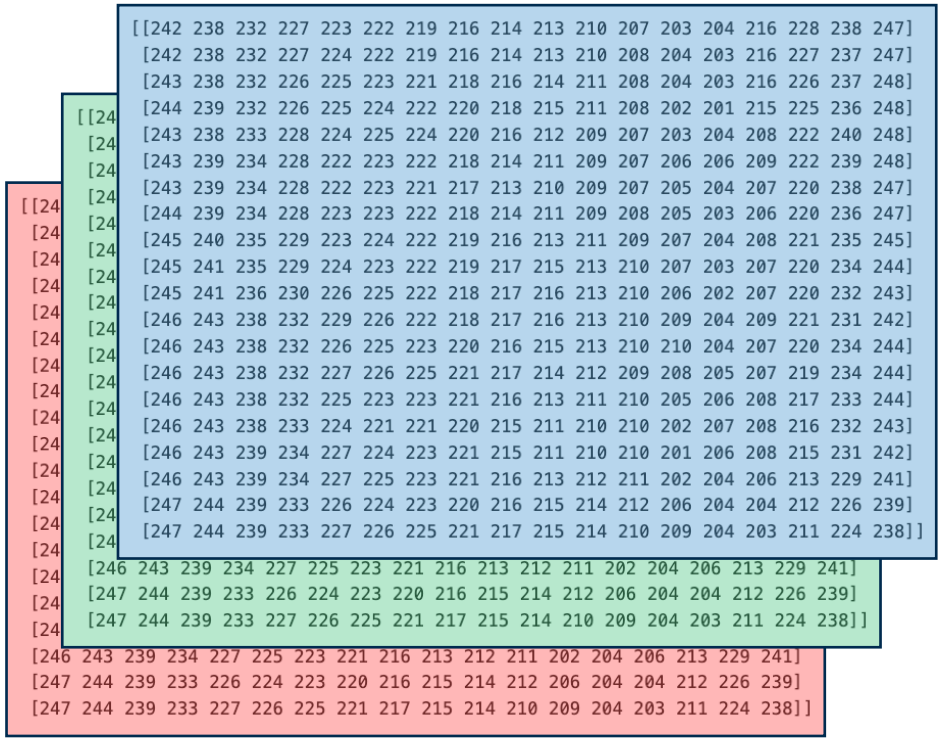


CRICOS No.00213J

Why is Robotic / Computer Vision Hard?



An Image is just an array of numbers …



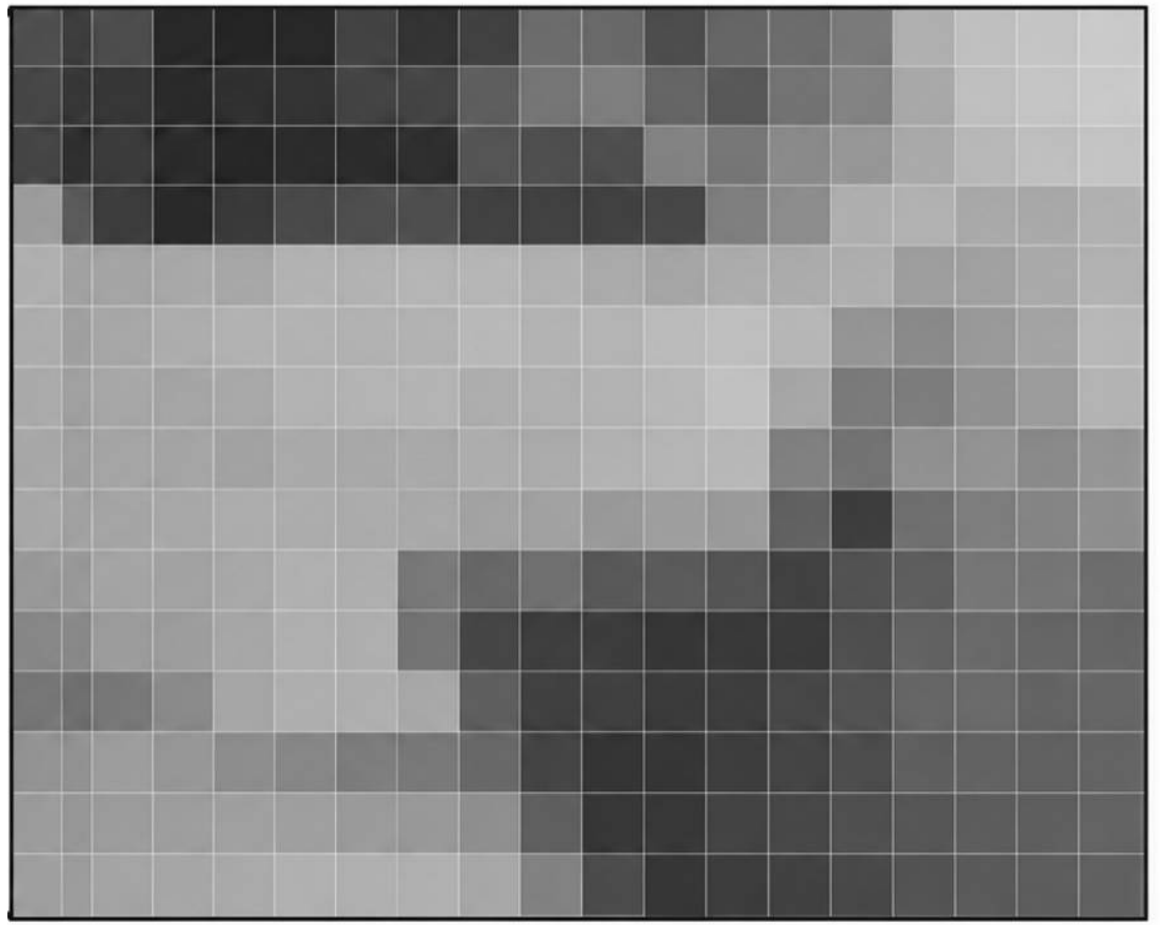
Colour Images: Red, Green, Blue channels



CRICOS No.00213J

An Image is just an array of numbers …

```
[[242 238 232 227 223 222 219 216 214 213 210 207 203 204 216 228 238 247]
 [242 238 232 227 224 222 219 216 214 213 210 208 204 203 216 227 237 247]
 [243 238 232 226 225 223 221 218 216 214 211 208 204 203 216 226 237 248]
 [244 239 232 226 225 224 222 220 218 215 211 208 202 201 215 225 236 248]
 [243 238 233 228 224 225 224 220 216 212 209 207 203 204 208 222 240 248]
 [243 239 234 228 222 223 222 218 214 211 209 207 206 206 209 222 239 248]
 [243 239 234 228 222 223 221 217 213 210 209 207 205 204 207 220 238 247]
 [244 239 234 228 223 223 222 218 214 211 209 208 205 203 206 220 236 247]
 [245 240 235 229 223 224 222 219 216 213 211 209 207 204 208 221 235 245]
 [245 241 235 229 224 223 222 219 217 215 213 210 207 203 207 220 234 244]
 [245 241 236 230 226 225 222 218 217 216 213 210 206 202 207 220 232 243]
 [246 243 238 232 229 226 222 218 217 216 213 210 209 204 209 221 231 242]
 [246 243 238 232 226 225 223 220 216 215 213 210 210 204 207 220 234 244]
 [246 243 238 232 227 226 225 221 217 214 212 209 208 205 207 219 234 244]
 [246 243 238 232 225 223 223 221 216 213 211 210 205 206 208 217 233 244]
 [246 243 238 233 224 221 221 220 215 211 210 210 202 207 208 216 232 243]
 [246 243 239 234 227 224 223 221 215 211 210 210 201 206 208 215 231 242]
 [246 243 239 234 227 225 223 221 216 213 212 211 202 204 206 213 229 241]
 [247 244 239 233 226 224 223 220 216 215 214 212 206 204 204 212 226 239]
 [247 244 239 233 227 226 225 221 217 215 214 210 209 204 203 211 224 238]]
```



Grayscale Images: only one channel

CRICOS No.00213J

Image Formats

- Images can come in different datatypes
- **8-bit Unsigned Integer** (uint8)
- **Floating Point** (e.g. float32, float16)

```
[[242 238 232 227 223]    [[0.94901961 0.93333333 0.90980392 0.89019608 0.8745098 ]
 [242 238 232 227 224]     [0.94901961 0.93333333 0.90980392 0.89019608 0.87843137]
 [243 238 232 226 225]     [0.95294118 0.93333333 0.90980392 0.88627451 0.88235294]
 [244 239 232 226 225]     [0.95686275 0.9372549  0.90980392 0.88627451 0.88235294]
 [243 238 233 228 224]     [0.95294118 0.93333333 0.91372549 0.89411765 0.87843137]
 [243 239 234 228 222]     [0.95294118 0.9372549  0.91764706 0.89411765 0.87058824]
 [243 239 234 228 222]     [0.95294118 0.9372549  0.91764706 0.89411765 0.87058824]
 [244 239 234 228 223]     [0.95686275 0.9372549  0.91764706 0.89411765 0.8745098 ]
 [245 240 235 229 223]     [0.96078431 0.94117647 0.92156863 0.89803922 0.8745098 ]
 [245 241 235 229 224]     [0.96078431 0.94509804 0.92156863 0.89803922 0.87843137]
 [245 241 236 230 226]     [0.96078431 0.94509804 0.9254902  0.90196078 0.88627451]
 [246 243 238 232 229]     [0.96470588 0.95294118 0.93333333 0.90980392 0.89803922]
 [246 243 238 232 226]     [0.96470588 0.95294118 0.93333333 0.90980392 0.88627451]
 [246 243 238 232 227]     [0.96470588 0.95294118 0.93333333 0.90980392 0.89019608]
 [246 243 238 232 225]     [0.96470588 0.95294118 0.93333333 0.90980392 0.88235294]
 [246 243 238 233 224]     [0.96470588 0.95294118 0.93333333 0.91372549 0.87843137]
 [246 243 239 234 227]     [0.96470588 0.95294118 0.9372549  0.91764706 0.89019608]
 [246 243 239 234 227]     [0.96470588 0.95294118 0.9372549  0.91764706 0.89019608]
 [247 244 239 233 226]     [0.96862745 0.95686275 0.9372549  0.91372549 0.88627451]
 [247 244 239 233 227]]    [0.96862745 0.95686275 0.9372549  0.91372549 0.89019608]]
```

- When working with images, be aware of the used datatype!

Working with Images

- There are a number of software libraries
  - **OpenCV** (Python & C++) https://opencv.org/ ← this is what we'll use
  - Scikit-image (Python) https://scikit-image.org/
  - Python Imaging Library (PIL / Pillow) https://pillow.readthedocs.io
  - Machine Vision Toolbox (Matlab & Python): https://github.com/petercorke/machinevision-toolbox-python









CRICOS No.00213J

Let’s look at some simple operations

- A first glimpse at OpenCV (if you have not used it before)
- Image Convolution (yes, as in Convolutional Neural Network)
- Understand the concept of the derivative of an image



Simple Image Operations with OpenCV

- Load an image

```
# load an image with OpenCV
import cv2 as cv
img = cv.imread('kitchen.jpeg', cv.IMREAD_GRAYSCALE)
```



Simple Image Operations with OpenCV

```
# load an image with OpenCV
import cv2 as cv
img = cv.imread('kitchen.jpeg', cv.IMREAD_GRAYSCALE)
```

```
type(img)
```

```
numpy.ndarray
```



Simple Image Operations with OpenCV

```
# load an image with OpenCV
import cv2 as cv
img = cv.imread('kitchen.jpeg', cv.IMREAD_GRAYSCALE)
```

```
type(img)
```

```
numpy.ndarray
```

```
img.shape
```

```
(1024, 1024)
```



Image Coordinates – Access to the Pixels

We often use u and v to specify coordinates in an image.

This is because x and y are already used as coordinates in 3D points (x,y,z)

To access a particular pixel in the image: img[v, u]

The notation is
- [vertical, horizontal]
- [row, column]



CRICOS No.00213J

Simple Image Operations with OpenCV

```
# load an image with OpenCV
import cv2 as cv
img = cv.imread('kitchen.jpeg', cv.IMREAD_GRAYSCALE)
```

```
img[432,102]
```

```
np.uint8(164)
```



Simple Image Operations with OpenCV

- Load as color image

```
# load an image with OpenCV
import cv2 as cv
img = cv.imread('kitchen.jpeg', cv.IMREAD_COLOR)
```

```
img.shape
```

```
(1024, 1024, 3)
```

```
img[432,102]
```

```
array([165, 167, 159], dtype=uint8)
```



CRICC

Simple Image Operations with OpenCV

- Thresholding: which pixels are brighter / darker than a threshold value?

```
# load an image with OpenCV
import cv2 as cv
img = cv.imread('kitchen.jpeg', cv.IMREAD_GRAYSCALE)

# find pixels that are very bright
```



Simple Image Operations with OpenCV

- Thresholding: which pixels are brighter / darker than a threshold value?

```
# load an image with OpenCV
import cv2 as cv
img = cv.imread('kitchen.jpeg', cv.IMREAD_GRAYSCALE)

# find pixels that are very bright
mask = img>220

# set all other pixels to black
```



Simple Image Operations with OpenCV

- Thresholding: which pixels are brighter / darker than a threshold value?

```
# load an image with OpenCV
import cv2 as cv
img = cv.imread('kitchen.jpeg', cv.IMREAD_GRAYSCALE)

# find pixels that are very bright
mask = img>220

# set all other pixels to black
img[~mask] = 0
```



Simple Image Operations

- Thresholding: which pixels are very red?

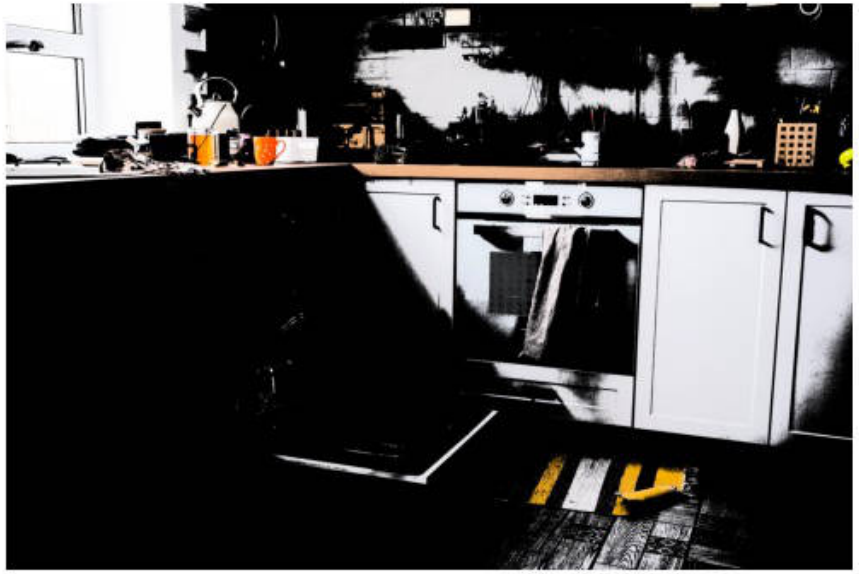
```
# load an image with OpenCV
import cv2 as cv
img = cv.imread('kitchen.jpeg', cv.IMREAD_COLOR)

# find pixels that are very red
mask = img[:,:,2] > 220

# set all other pixels to black
img[~mask,:] = 0
```

- Why are the white pixels not set to black?





CRICOS No.00213J

Simple Image Operations

- Thresholding: which pixels are very red?

```
# load an image with OpenCV
import cv2 as cv
import numpy as np
img = cv.imread('kitchen.jpeg', cv.IMREAD_COLOR)

# find pixels that are very red
intensity = np.sum(img, axis=2)
mask = img[:,:,2] / intensity > 0.5

# set all other pixels to black
img[~mask,:] = 0
```

- Why are the white pixels not set to black?
  - Use chromaticity: r = R / (R+G+B)



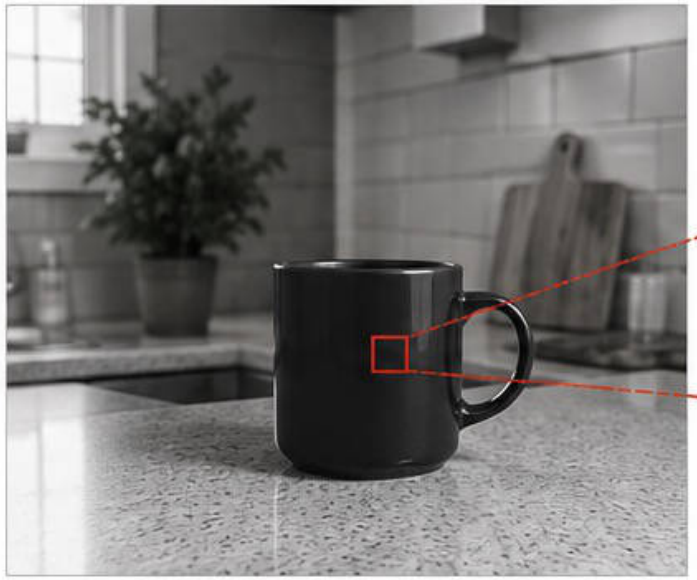


CRICOS No.00213J

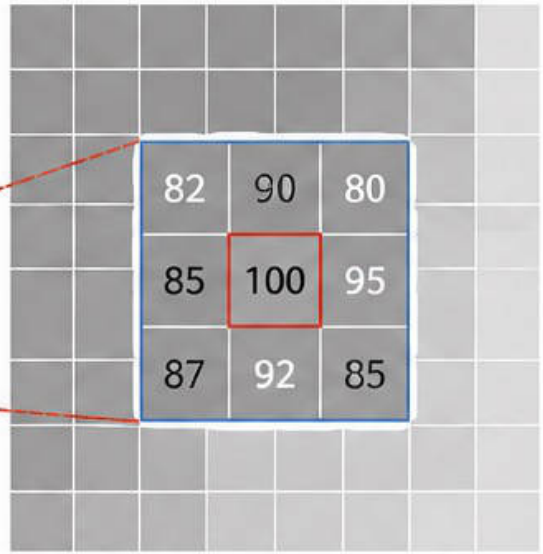
Operations on Pixel Neighbourhoods

Idea: Replace each pixel with a new value that depends on the pixels around it

1. Original image



2. Look at surrounding pixels



3. Compute new value

Example: 3×3 average (blur)

$$\frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix} = 88$$

New value = average of the 3×3 neighborhood

(82 + 90 + 80 + 85 + 100 + 95 + 87 + 92 + 85) / 9 = 88

Illustration by GPT5.5

Operations on Pixel Neighbourhoods

- Replace each pixel with a new value depending on the pixels around it

```
# load an image with OpenCV
import cv2 as cv
import numpy as np
img = cv.imread('kitchen.jpeg', cv.IMREAD_COLOR)

# do a Gaussian filter of the image
img2 = cv.GaussianBlur(img, (25, 25),0 )
```

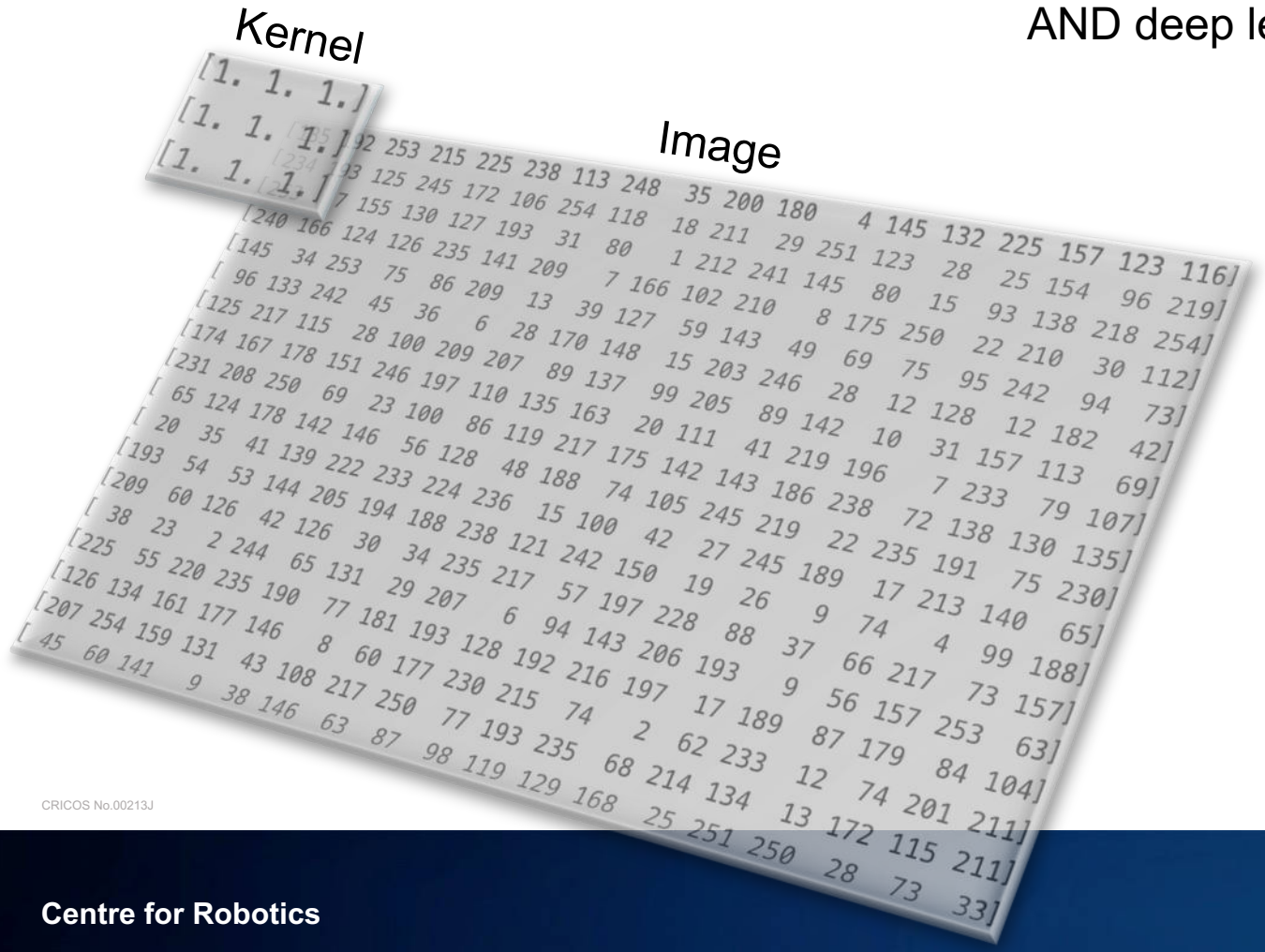
- Check out OpenCV documentation and Prac notebooks for more examples.





Operations on Neighbourhoods: Image Convolution

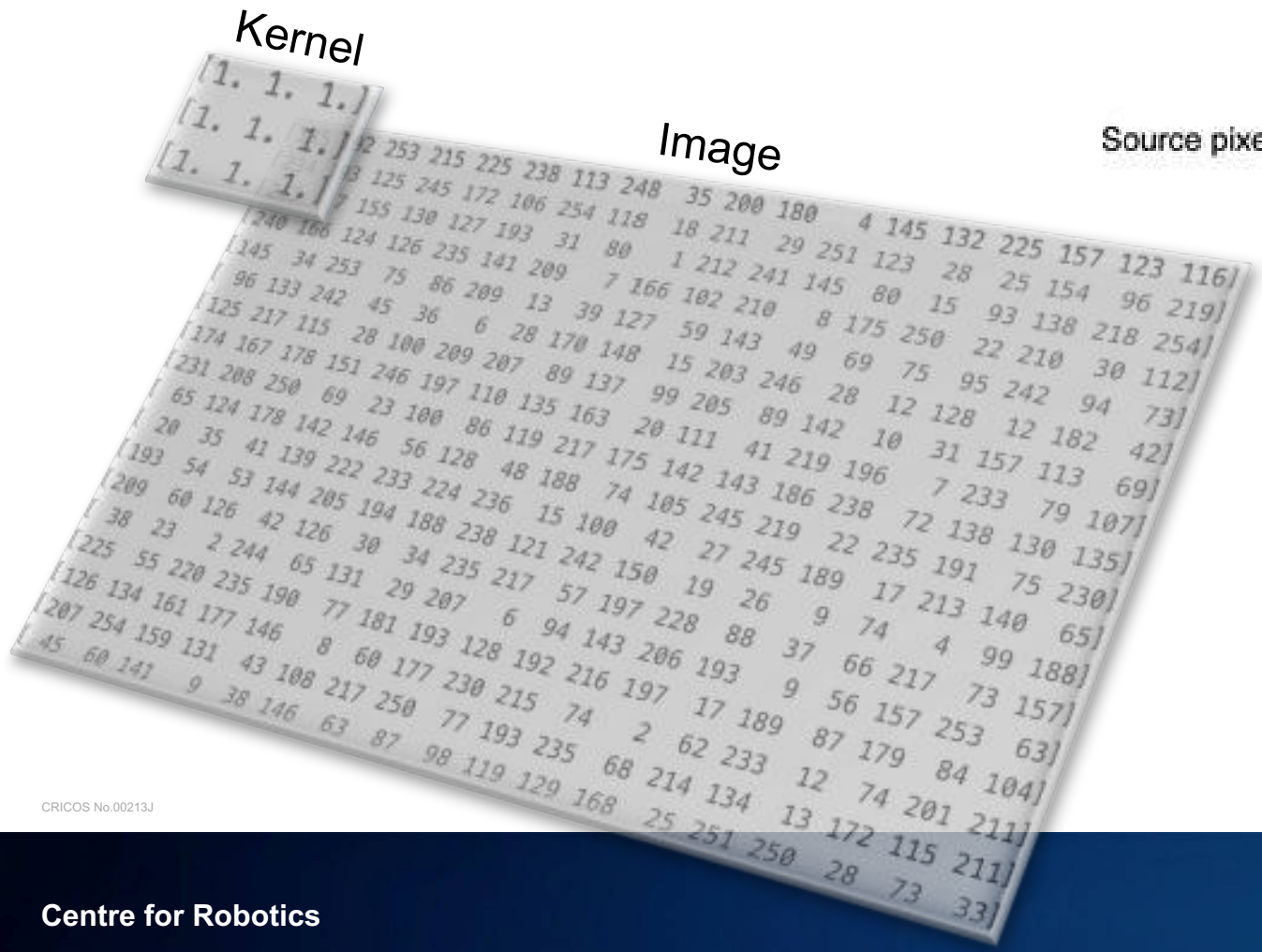
Important Operation underpinning traditional image processing AND deep learning (convolutional neural networks)

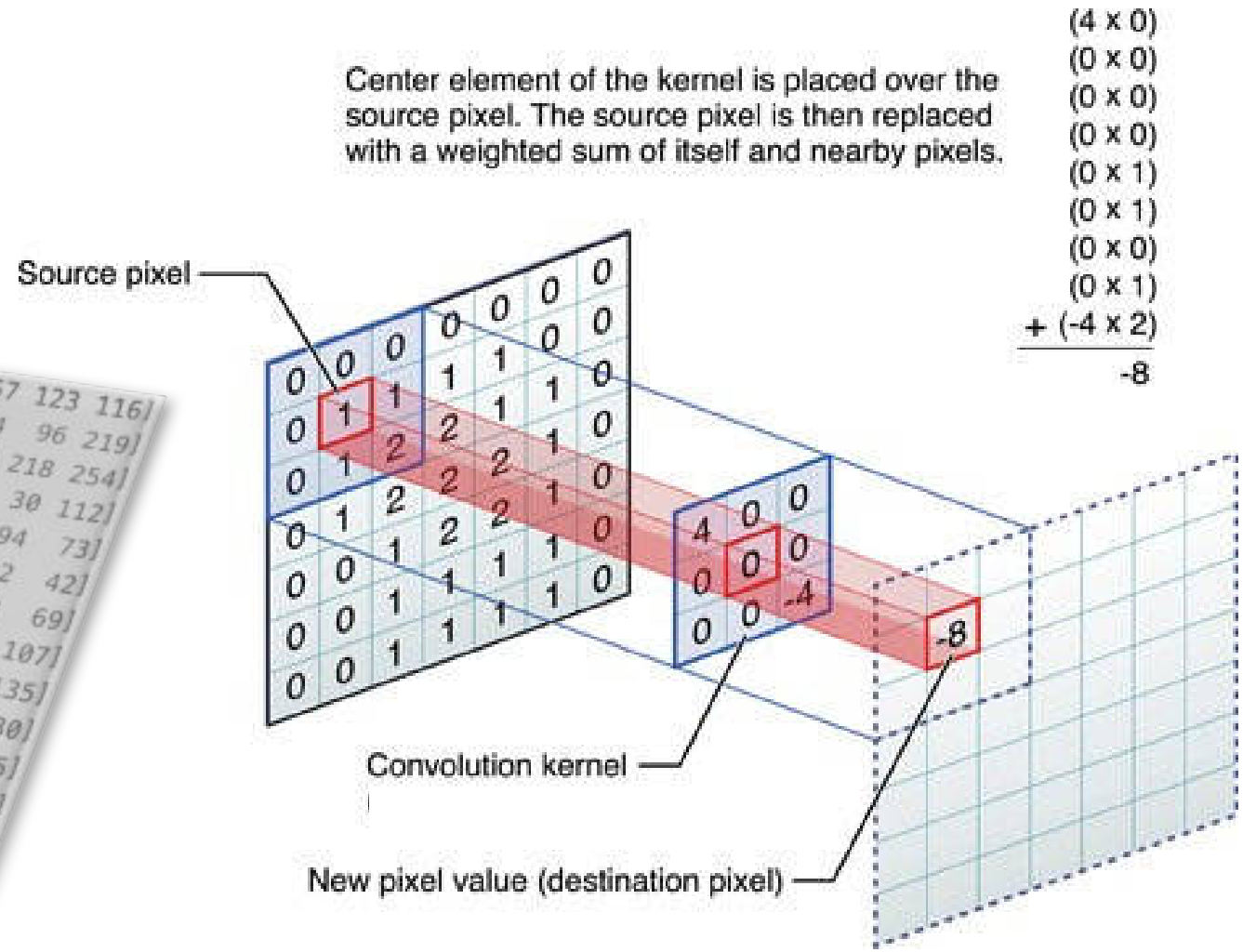


https://github.com/vdumoulin/conv_arithmetic/

CRICOS No.00213J

Image Convolution

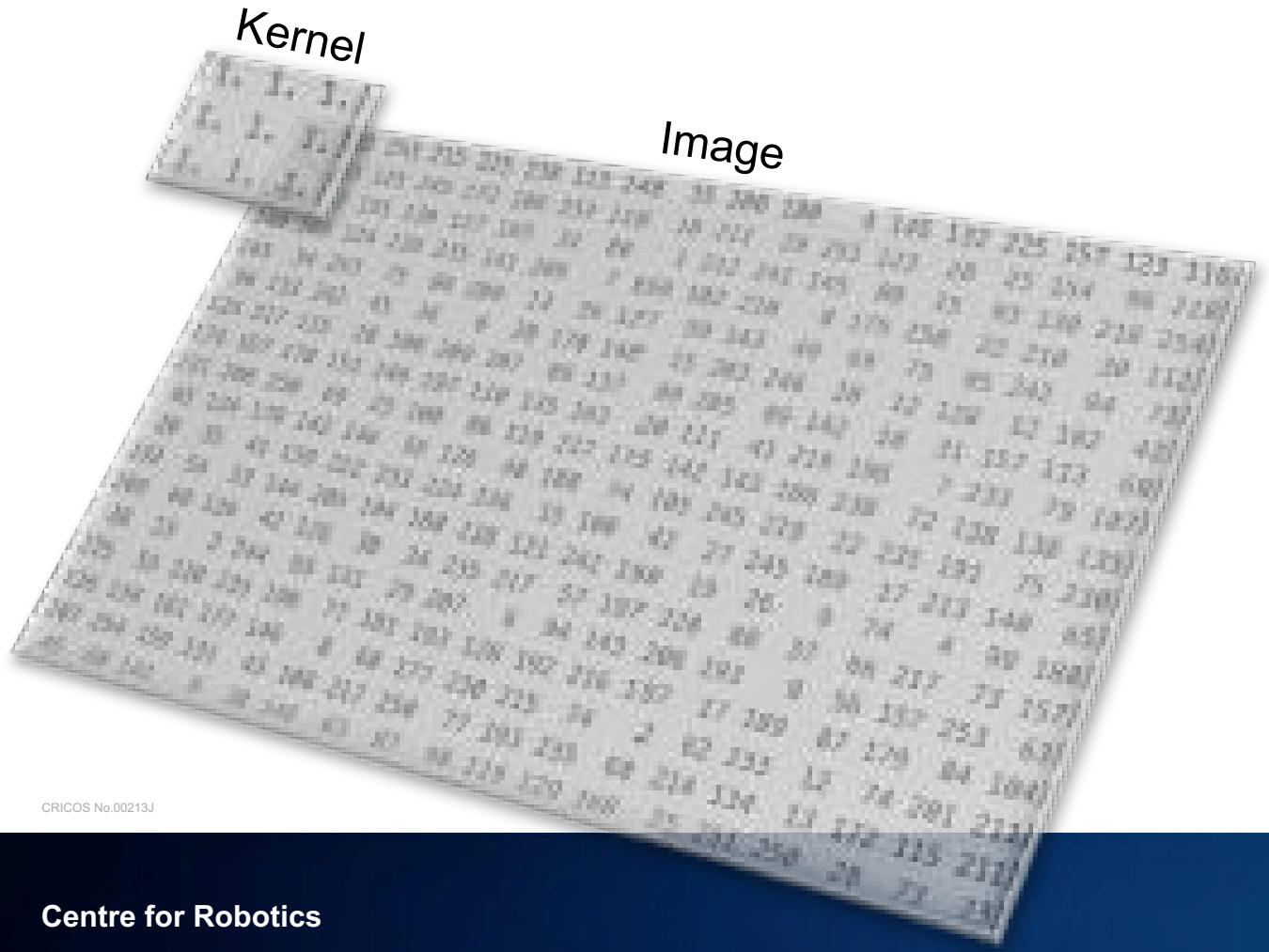


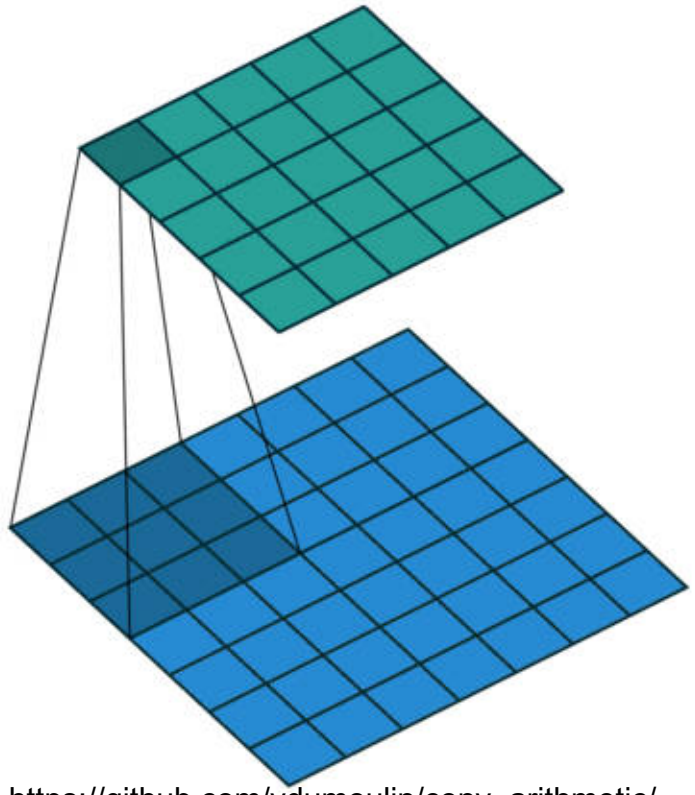


CRICOS No.00213J

Image credit: https://mriquestions.com/what-is-convolution.html

Image Convolution





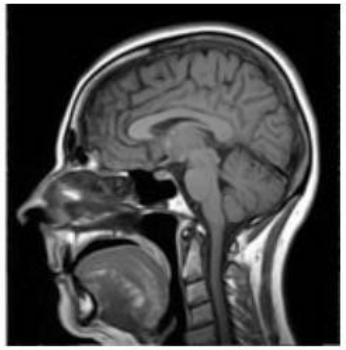
https://github.com/vdumoulin/conv_arithmetic/

CRICOS No.00213J

Image Convolution

- Different kernels lead to very different results
- Traditionally “hand-designed”
- Convolutional Networks: kernels are learned from data

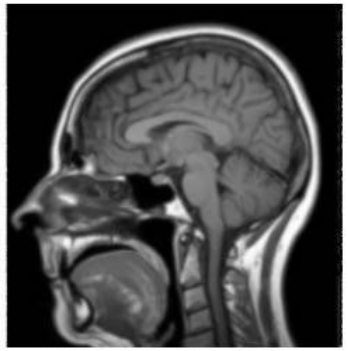
0	0	0
0	1	0
0	0	0



Identity Kernel: Just the starting image itself.

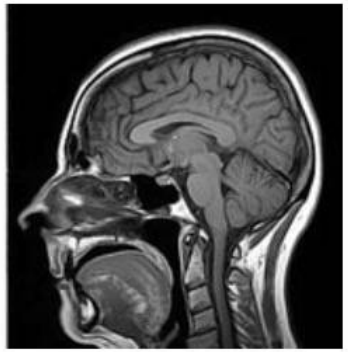
$\frac{1}{9}$

1	1	1
1	1	1
1	1	1



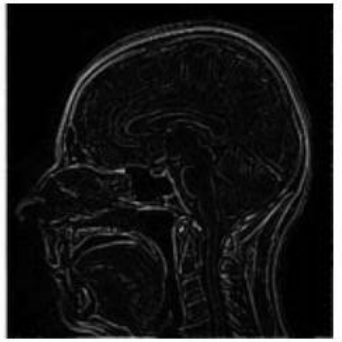
Box Blur Kernel: Each pixel is averaged equally with its 8 neighbors.

0	-1	0
-1	5	-1
0	-1	0



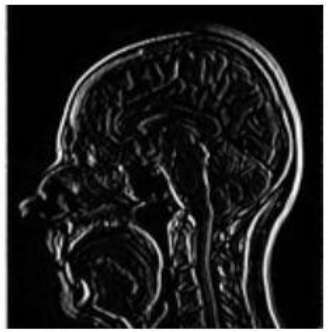
Sharpen Kernel:: Differences emphasized with its adjacent pixel values.

-1	-1	-1
-1	8	-1
-1	-1	-1



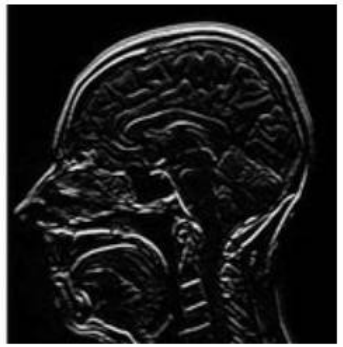
Outline ("Edge") Kernel: Highlights neighboring pixels with large differences of intensity.

1	0	-1
2	0	-2
1	0	-1



Left-to-right Sobel Kernel:. Used to accentuate differences between pixels along the horizontal axis

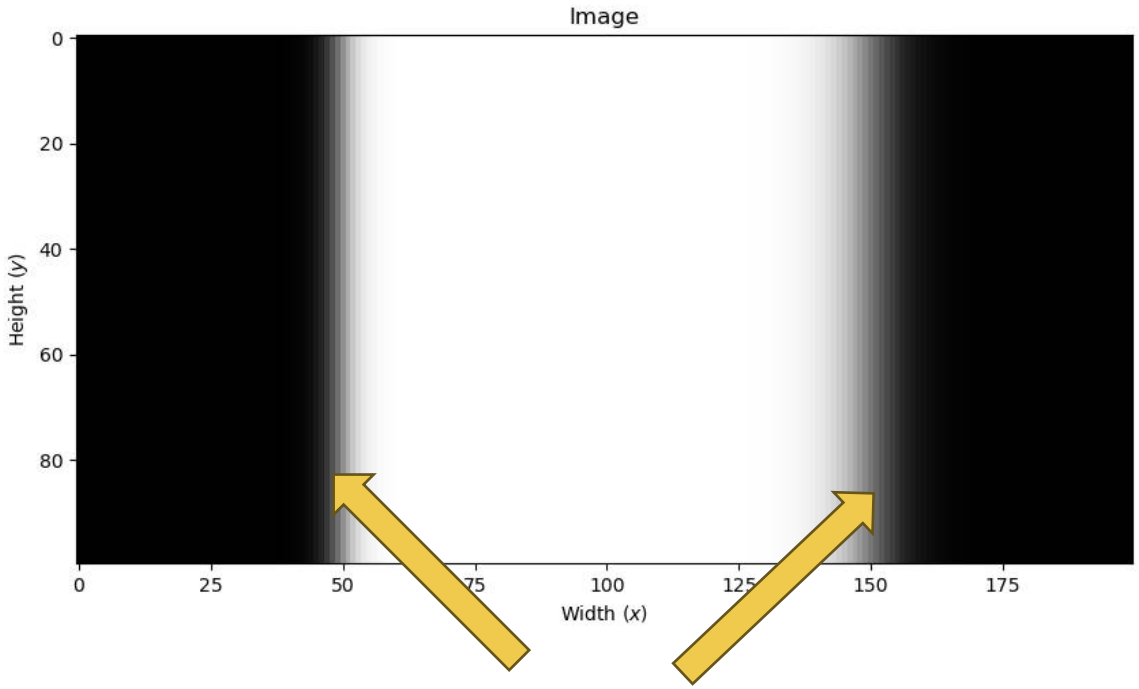
-1	-2	-1
0	0	0
1	2	1



Superior-to-inferior Sobel Kernel: Used to accentuate differences between pixels along the vertical axis.

Image credit: https://mriquestions.com/what-is-convolution.html

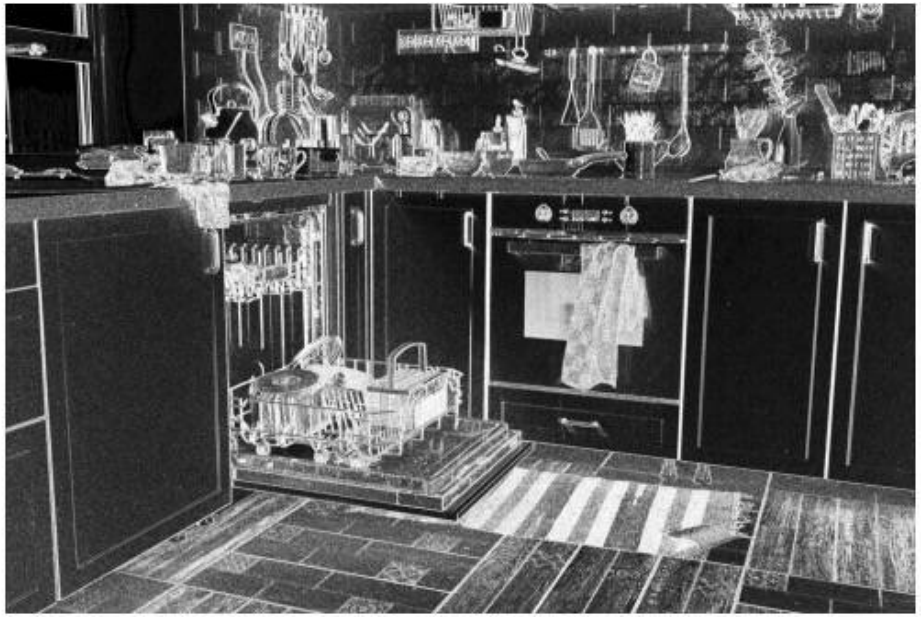
Edge Detection



Edges

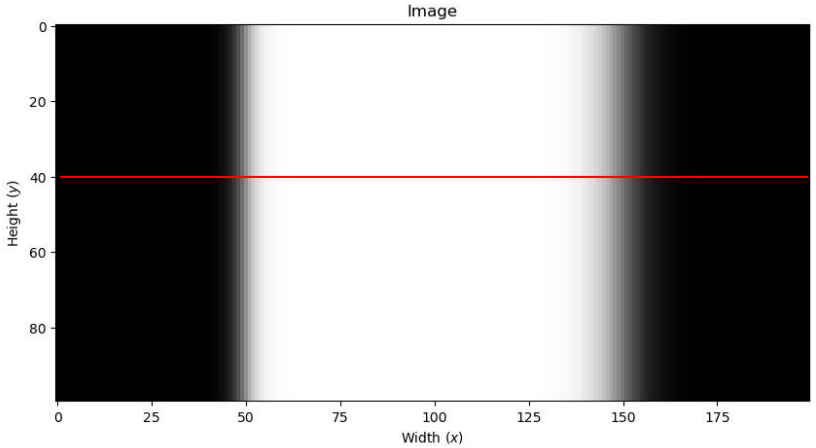
Intensity changes rapidly.

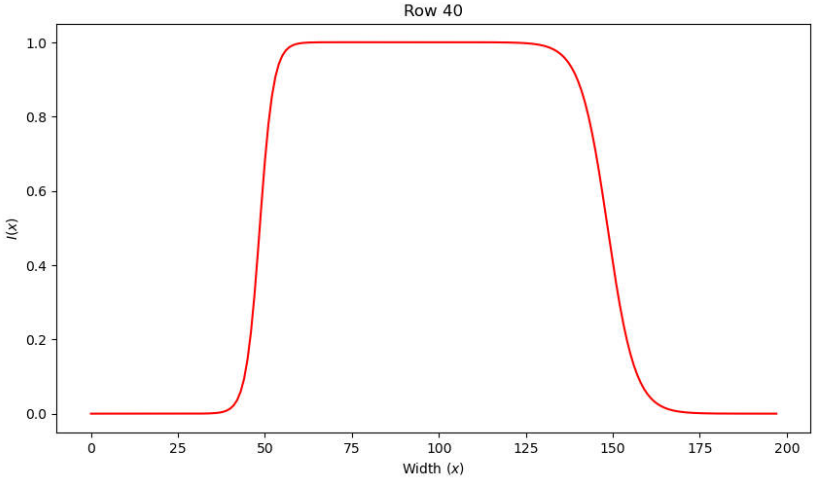




CRICOS No.00213J

Edge Detection – Idea: Use Derivatives!

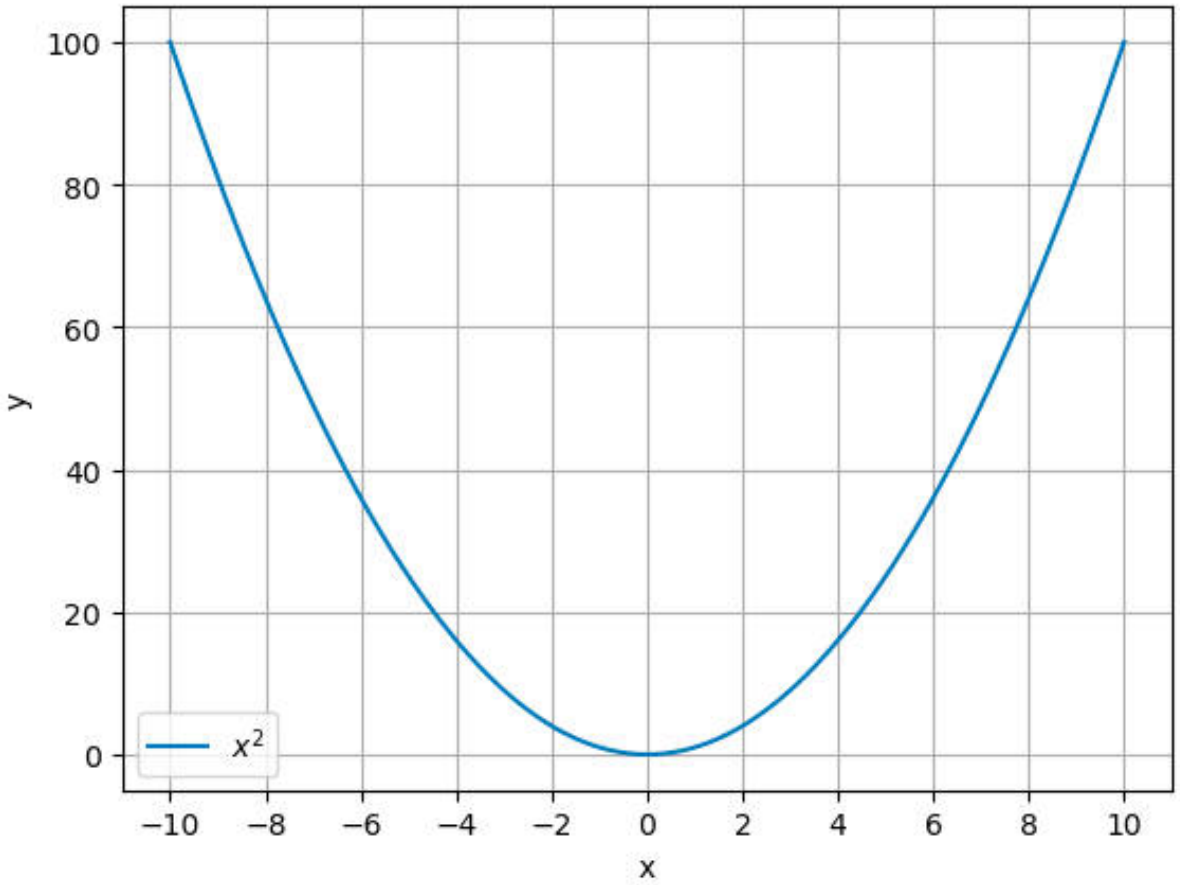




How can we calculate the derivative of an image?

It is a two-dimensional function: f(x,y)

Edge Detection & Image Derivatives

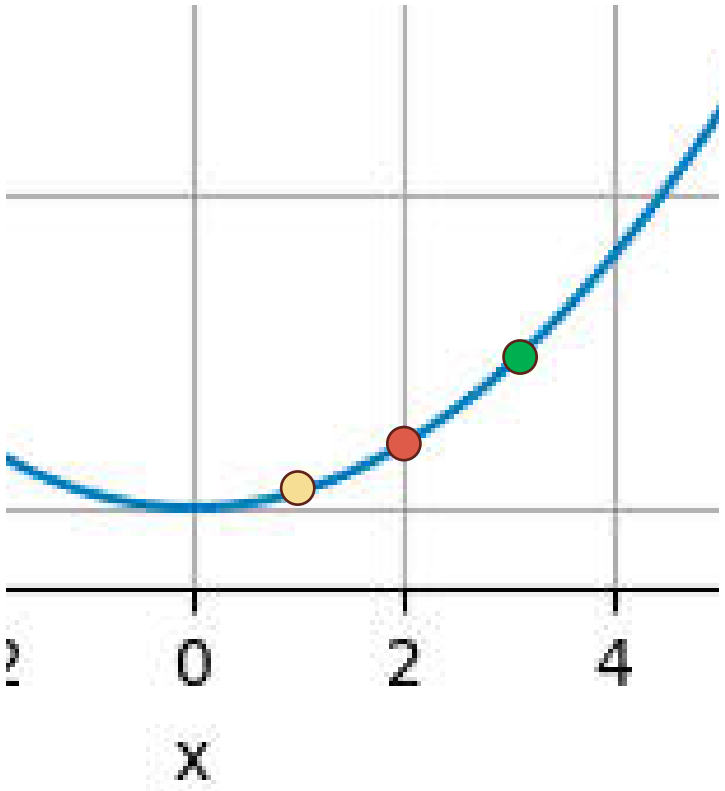


Let's **simplify**!

What even is the derivative of a function?

How can we calculate the derivative of a 1D function f(x)?

Edge Detection & Image Derivatives



Simplify: How to calculate the derivative of a 1D function f(x)?

Central finite differences method:

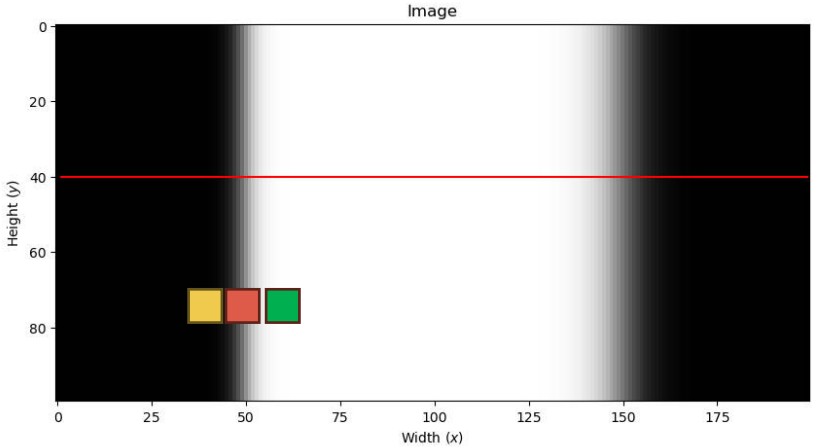
$$\frac{\partial f}{\partial x} \approx \frac{f(x+\epsilon) - f(x-\epsilon)}{2\epsilon}$$

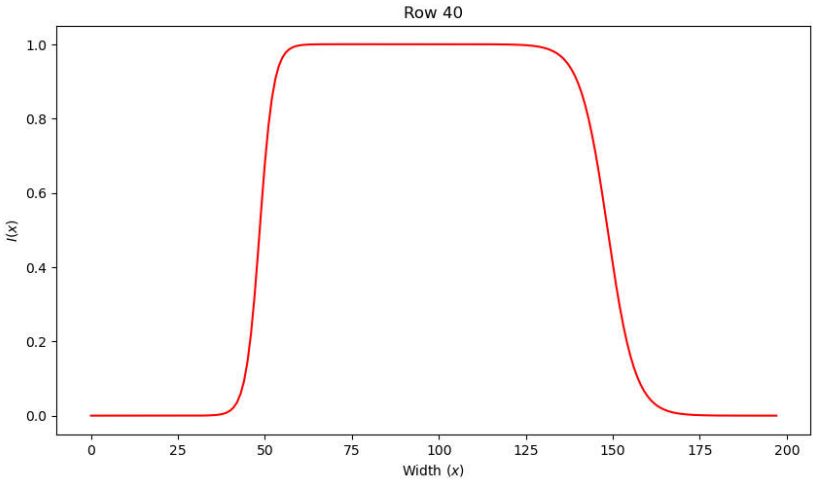
E.g. at $x = 2$ with $\epsilon = 1$

$$\frac{f(2+1) - f(2-1)}{2} = \frac{9-1}{2} = 4$$

CRICOS No.00213J

Edge Detection & Image Derivatives





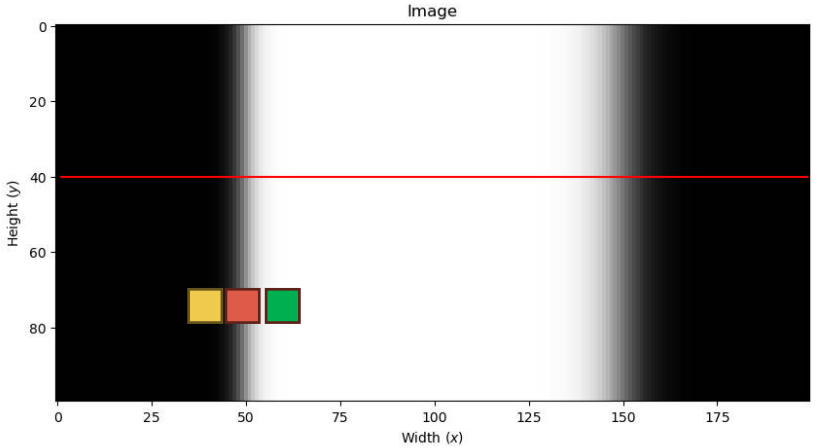
How can we calculate the derivative of an image?

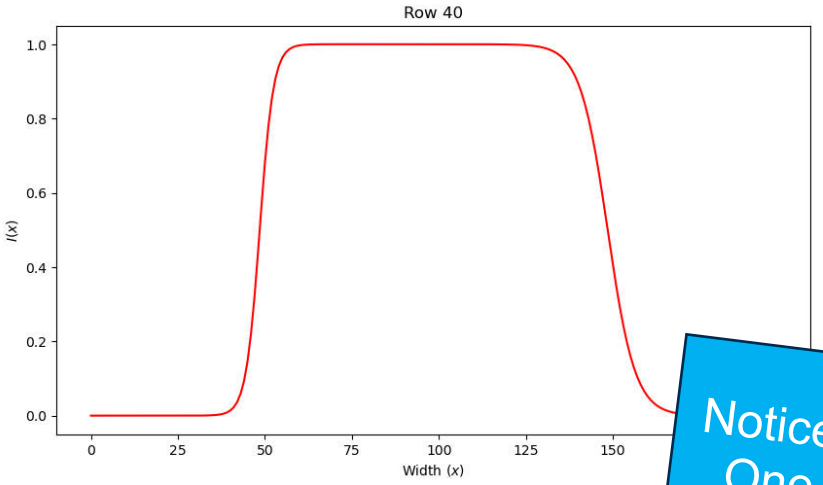
Same principle: Central Finite Differences

$$\frac{\partial I(x,y)}{\partial x} \approx \frac{I(x+1,y) - I(x-1,y)}{2}$$

$$\frac{\partial I(x,y)}{\partial y} \approx \frac{I(x,y+1) - I(x,y-1)}{2}$$

Edge Detection & Image Derivatives





How can we calculate the derivative of an image?

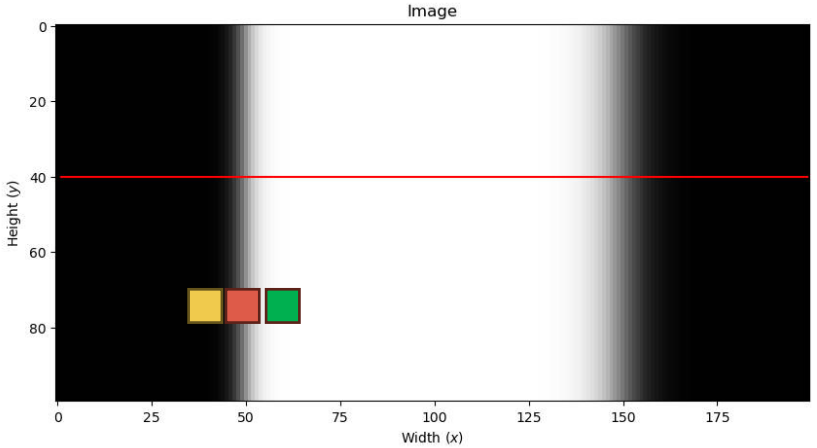
Same principle: Central Finite Differences

$$\frac{\partial I(x,y)}{\partial x} \approx \frac{I(x+1,y) - I(x-1,y)}{2}$$

$$\frac{\partial I(x,y)}{\partial y} \approx \frac{I(x,y+1) - I(x,y-1)}{2}$$

Notice how there are two partial derivatives!
One in x-direction, and one in y-direction!

Edge Detection & Image Derivatives



How can we calculate the derivative of an image?

Same principle: Central Finite Differences

$$\frac{\partial I(x,y)}{\partial x} \approx \frac{I(x+1,y) - I(x-1,y)}{2}$$

$$\frac{\partial I(x,y)}{\partial y} \approx \frac{I(x,y+1) - I(x,y-1)}{2}$$

Prewitt filter

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

We can do this with a **convolution** operation!

CRICOS No.00213J

Image Gradient

- The **image gradient**: $\nabla \mathrm{I} = \left[\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y}\right]$

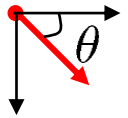
- The gradient points in the **direction of steepest ascent**:



$\nabla \mathrm{I} = \left[\frac{\partial I}{\partial x}, 0\right]$



$\nabla \mathrm{I} = \left[0, \frac{\partial I}{\partial y}\right]$



$\nabla \mathrm{I} = \left[\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y}\right]$

- The **gradient direction** (perpendicular to the edge) is given by: $\theta = atan2\left(\frac{\partial I}{\partial y}, \frac{\partial I}{\partial x}\right)$

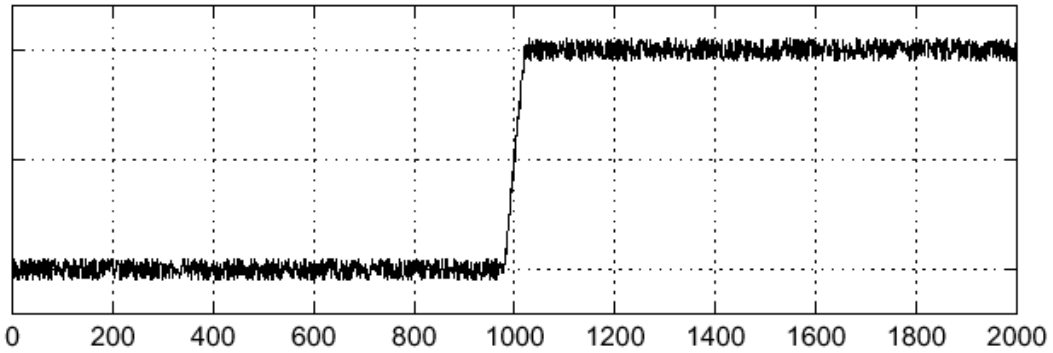
- The **edge strength** is given by the gradient magnitude $\|\nabla I\| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}$

Slide courtesy Davide Scaramuzza

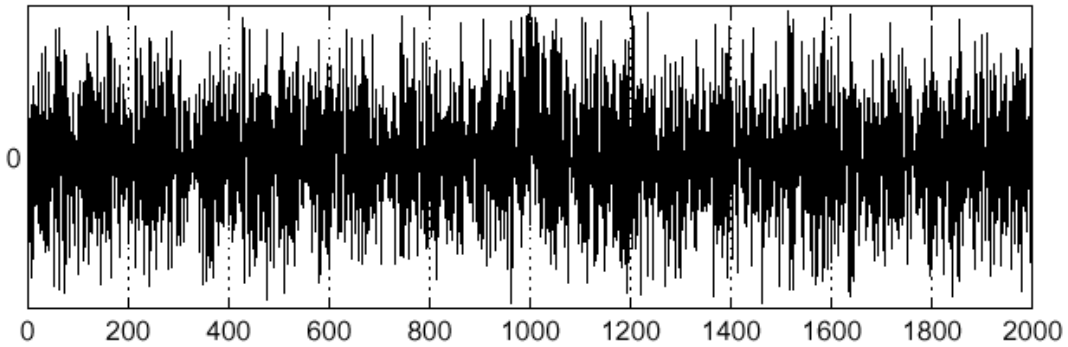
Effects of Noise

- Consider a single row or column of the image

$I(x)$



$\frac{\partial I(x)}{\partial x}$



Where is the edge?

Slide courtesy Davide Scaramuzza

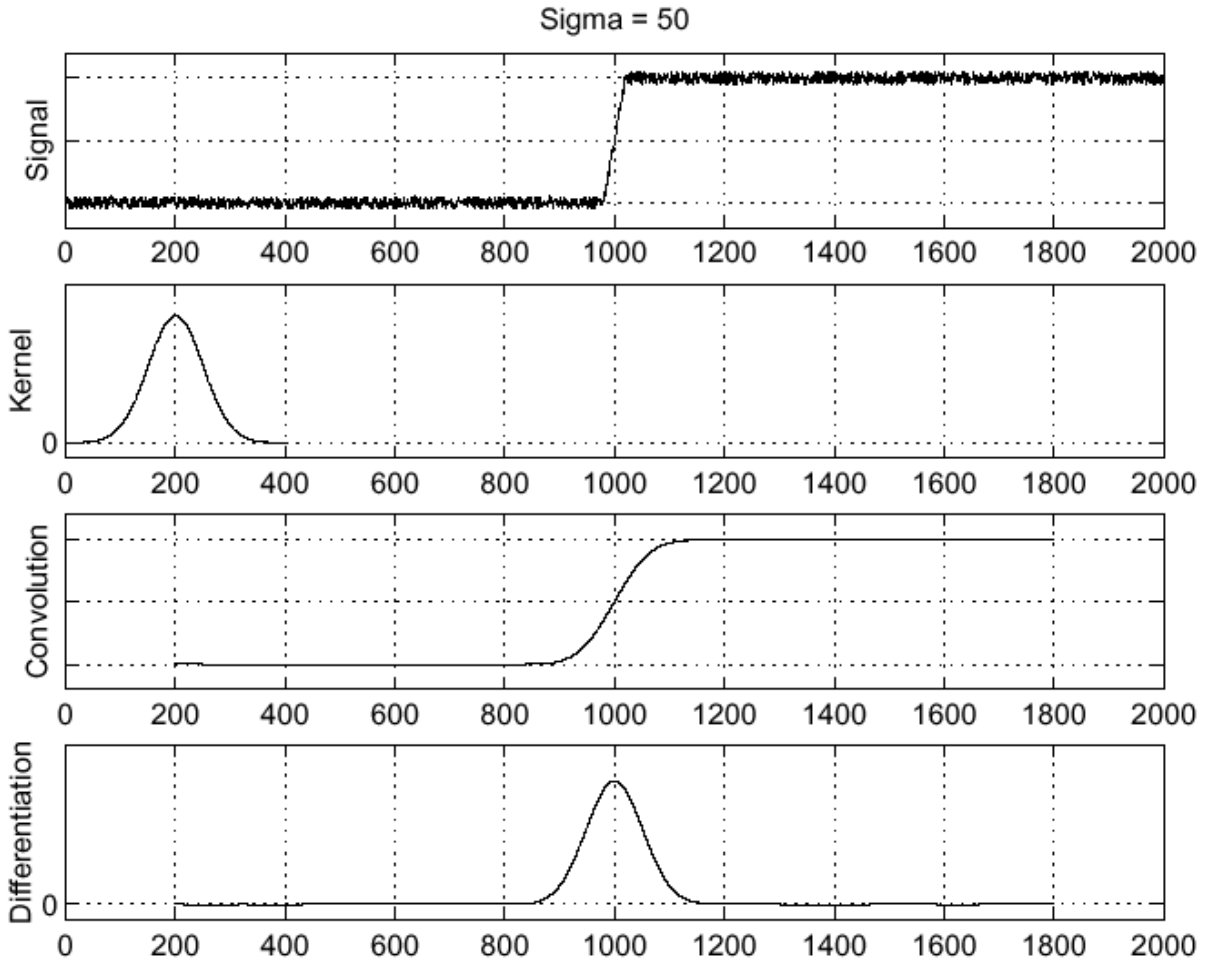
Solution: smooth first

I

H

$I * H$

$\frac{\partial}{\partial x}(I * H)$



Location of edges: **look for peaks** in $\frac{\partial}{\partial x}(I * H)$

Slide courtesy Davide Scaramuzza

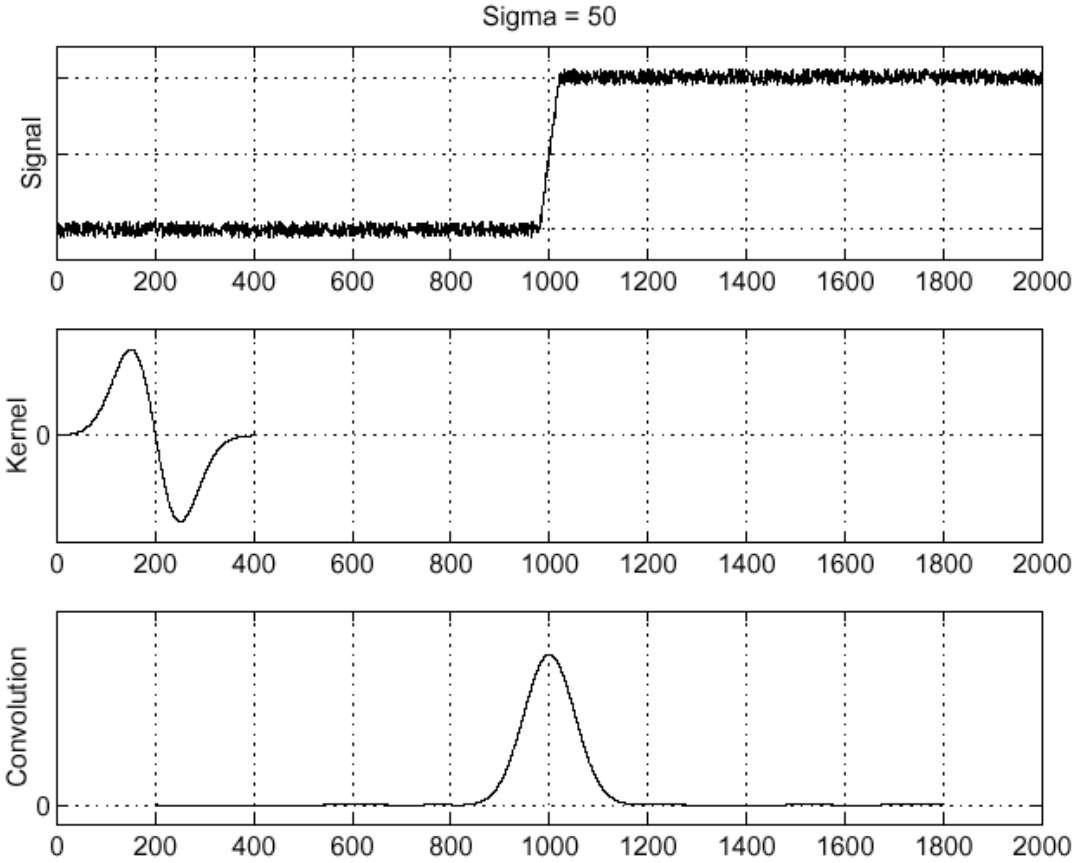
Alternative: combine derivative and smoothing filter

- Differentiation property of convolution: $\frac{\partial}{\partial x}(I * H) = I * \frac{\partial H}{\partial x}$

I

$\frac{\partial H}{\partial x}$

$I * \frac{\partial H}{\partial x}$



Slide courtesy Davide Scaramuzza

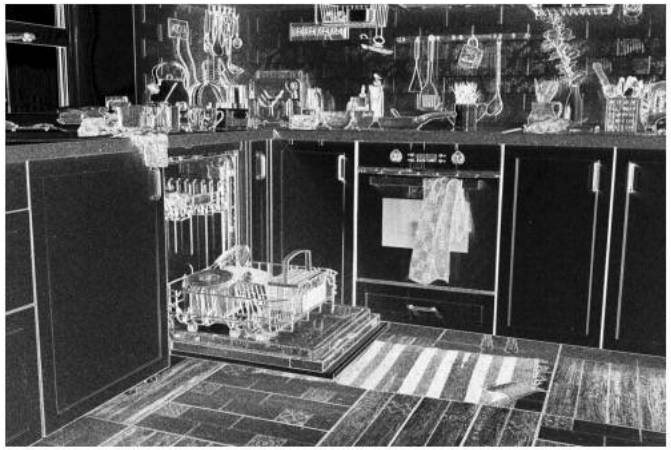
Edge Detection & Image Derivatives

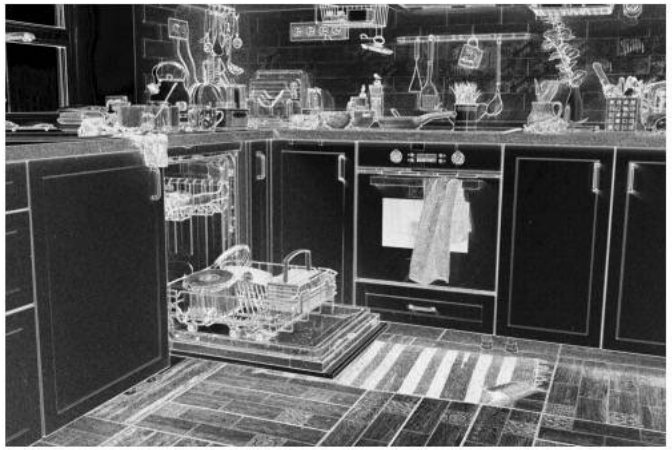


Sobel filter

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad , \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$





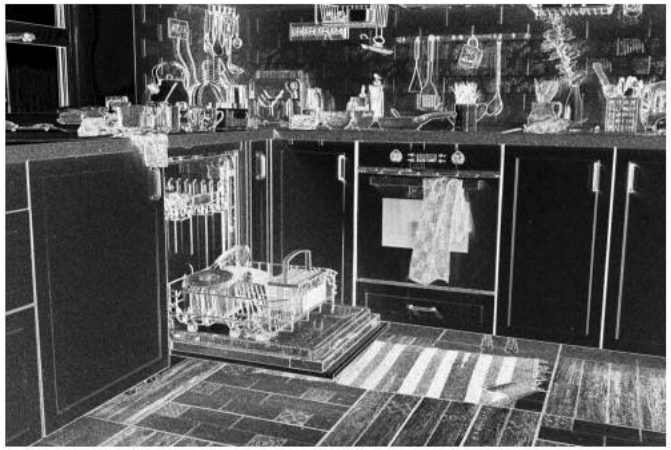


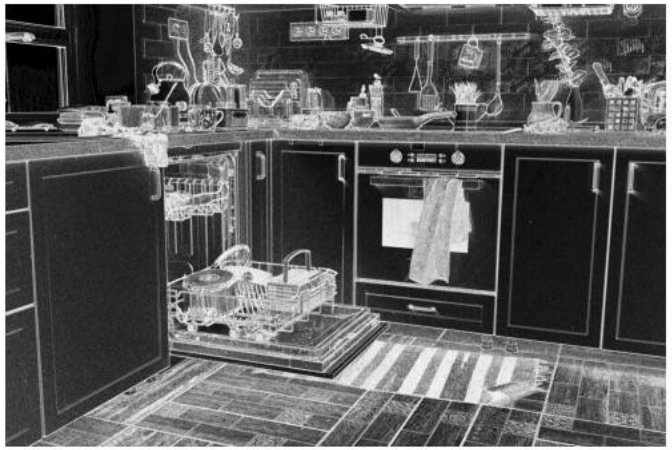
Edge Detection & Image Derivatives

```
# sobel edge detection
img = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
grad_x = cv.Sobel(img, cv.CV_64F, 1, 0, ksize=5)
grad_y = cv.Sobel(img, cv.CV_64F, 0, 1, ksize=5)
abs_grad_x = cv.convertScaleAbs(grad_x)
abs_grad_y = cv.convertScaleAbs(grad_y)
grad = cv.addWeighted(abs_grad_x, 0.5, abs_grad_y, 0.5, 0)
```









Recap

Recap

- What is Robotic Vision?
- Images are arrays of pixels
  - Simple operations: thresholding, convolutions, edge detection
- Advise: Focus on the ideas, don’t get lost in details
- Build your toolbox

CRICOS No.00213J

Next Week

How can we find distinctive points in an image and recognize the same point again in another image or the next image in a video sequence?



CRICOS No.00213J